GUÍA PRÁCTICA

C# Windows Forms + SQL Server

Ejercicios con ADO.NET, consultas SQL y casos de prueba

18 ejercicios progresivos · Fechas dinámicas · Proyecto final

Tecnologías utilizadas

C# · Windows Forms · Visual Studio
SQL Server Express / LocalDB · ADO.NET
SqlConnection · SqlCommand · SqlDataReader · SqlDataAdapter · DataTable

Sin Entity Framework · Sin LINQ · Sin ORM · Sin soluciones: resuélvelo tú
Fecha de referencia de los casos de prueba: 30/09/2026

Índice

1. Objetivos de la guía 3

2. Requisitos previos 4

3. Configuración de SQL Server 5

4. Nivel 1 — Ejercicios básicos 7

EJERCICIO 1 — SELECT Y CONEXIÓN A SQL SERVER 8

EJERCICIO 2 — INSERT — REGISTRAR PRODUCTOS 11

EJERCICIO 3 — UPDATE — ACTUALIZAR CLIENTES 14

EJERCICIO 4 — DELETE — ELIMINAR PROVEEDORES 17

EJERCICIO 5 — CRUD COMPLETO DE EMPLEADOS 20

5. Nivel 2 — Ejercicios intermedios 24

EJERCICIO 6 — BÚSQUEDA POR NOMBRE (LIKE) 25

EJERCICIO 7 — BÚSQUEDA POR APELLIDO Y OTROS CAMPOS 28

EJERCICIO 8 — FILTROS POR RANGO DE FECHAS 31

EJERCICIO 9 — FUNCIONES DE AGREGACIÓN: SUM, COUNT, AVG, MIN Y MAX 34

EJERCICIO 10 — TOP Y ORDER BY: RANKINGS 38

EJERCICIO 11 — FILTROS MÚLTIPLES COMBINADOS 42

6. Nivel 3 — Intermedio/Avanzado y módulo de fechas 46

EJERCICIO 12 — INNER JOIN: CLIENTES Y VENTAS 47

EJERCICIO 13 — GROUP BY: ESTADÍSTICAS POR GRUPO 51

EJERCICIO 14 — HAVING: FILTRAR GRUPOS 56

EJERCICIO 15 — CONSULTAS POR PERÍODOS DE TIEMPO 61

EJERCICIO 16 — REPORTES Y ESTADÍSTICAS POR PERÍODO 66

7. Nivel 4 — Ejercicios avanzados y proyecto final 71

EJERCICIO 17 — SISTEMA CON MÚLTIPLES FILTROS Y REPORTES 72

EJERCICIO 18 — PROYECTO FINAL: SISTEMA DE GESTIÓN COMERCIAL 78

8. Matriz de seguimiento de casos de prueba 85

9. Checklist final 86

1. Objetivos de la guía

Esta guía no es un libro de teoría: es un cuaderno de **problemas para resolver**. Cada ejercicio te entrega el contexto, la base de datos, los datos de prueba, el diseño del formulario, los requerimientos y los **casos de prueba con resultados exactos**. Tú escribes el código C# y las consultas SQL, y compruebas que tu programa coincide con lo esperado.

Al terminar serás capaz de desarrollar por tu cuenta una aplicación Windows Forms conectada a SQL Server que pueda:

- Registrar, editar y eliminar datos (CRUD) con parámetros SQL.
- Buscar datos por texto (LIKE) y filtrarlos con múltiples condiciones.
- Trabajar con fechas: rangos, año, mes, año actual, mes actual, períodos anteriores y períodos relativos (últimos 7 y 30 días, 10 y 12 meses).
- Calcular estadísticas con COUNT, SUM, AVG, MIN y MAX, y obtener rankings con TOP y ORDER BY.
- Agrupar información con GROUP BY y HAVING, y relacionar tablas con INNER JOIN (incluido un JOIN de cuatro tablas).
- Generar reportes en un DataGridView, validar la información del usuario y manejar errores (SqlException, conexión, conversión).
- Comprobar que tu programa funciona mediante casos de prueba con resultados numéricos exactos.

Estructura de la guía

Capítulo	Ejercicios	Contenido
Nivel 1 — Básico	1 a 5	SELECT y conexión, INSERT, UPDATE, DELETE, CRUD completo
Nivel 2 — Intermedio	6 a 11	LIKE, búsqueda por apellido y otros campos, rango de fechas, agregados, TOP/ORDER BY, filtros múltiples
Nivel 3 — Intermedio/Avanzado	12 a 16	INNER JOIN, GROUP BY, HAVING, **módulo de fechas** (períodos y reportes)
Nivel 4 — Avanzado	17 y 18	Sistema con múltiples filtros y reportes (4 tablas) y proyecto final integrador
Cierre	—	Matriz de seguimiento de casos de prueba y checklist final

Cómo trabajar cada ejercicio

1. Ejecuta el script SQL del ejercicio (crea la base y los datos).
2. Crea el formulario con los controles indicados (mismos nombres).
3. Lee los requerimientos y las validaciones; escribe primero cada consulta en SSMS y compárala con los resultados esperados.
4. Pásala a C# con SqlConnection / SqlCommand / SqlDataAdapter y parámetros.
5. Ejecuta los casos de prueba EN ORDEN y marca los que pasan en la matriz final.
6. Si un caso falla, revisa tus datos: el resultado esperado está calculado con exactitud a partir de los INSERT.
7. Intenta el reto adicional y anota en el espacio de notas lo que aprendiste.

2. Requisitos previos

Área	Debes saber
C#	Variables, tipos, condicionales y ciclos; métodos; try/catch; conversión de tipos (TryParse); `using`.
Windows Forms	Crear un formulario, colocar controles, asignar nombres, manejar eventos Click y Load, usar DataGridView.
SQL	SELECT, WHERE, ORDER BY, INSERT, UPDATE, DELETE, tipos de datos básicos, llaves primarias y foráneas.
Herramientas	Visual Studio y SQL Server Management Studio (o el Explorador de objetos de SQL Server de Visual Studio).

Tecnologías permitidas

C#, Windows Forms, Visual Studio, SQL Server (Express o LocalDB), ADO.NET: **SqlConnection, SqlCommand, SqlDataReader, SqlDataAdapter, DataTable**; controles **DataGridView, TextBox, ComboBox, DateTimePicker, Button, Label** y **MessageBox**.

Tecnologías NO permitidas

Entity Framework, Entity Framework Core, LINQ, Dapper, ningún ORM y ninguna API externa. El objetivo es practicar directamente **Windows Forms + ADO.NET + SQL Server + SQL**.

3. Configuración de SQL Server

3.1 Instalar lo necesario

1. **Visual Studio** con la carga de trabajo «Desarrollo de escritorio con .NET». Marca también «Almacenamiento y procesamiento de datos» para incluir SQL Server Express LocalDB.
2. Opcional: **SQL Server Express** y **SQL Server Management Studio (SSMS)** desde el sitio oficial de Microsoft.
3. Comprueba LocalDB desde una consola: `sqllocaldb info`. Si la instancia no está iniciada: `sqllocaldb start MSSQLLocalDB`.

3.2 Cadenas de conexión (sustituye el nombre de la base del ejercicio)

Motor	Cadena de conexión
LocalDB	`Server=(localdb)\MSSQLLocalDB;Database=Ej01_Empleados;Integrated Security=True;TrustServerCertificate=True;`
SQL Server Express	`Server=.\SQLEXPRESS;Database=Ej01_Empleados;Integrated Security=True;TrustServerCertificate=True;`
Con usuario y clave	`Server=.\SQLEXPRESS;Database=Ej01_Empleados;User Id=usuario;Password=clave;TrustServerCertificate=True;`

Paquete ADO.NET: en .NET Framework usa el espacio de nombres `System.Data.SqlClient`; en .NET 6 o superior instala el paquete NuGet `Microsoft.Data.SqlClient` (mismas clases: SqlConnection, SqlCommand...). Este último cifra por defecto, por eso se incluye `TrustServerCertificate=True` en desarrollo local.

3.3 Ejecutar un script de los ejercicios

- **SSMS:** conéctate a `(localdb)\MSSQLLocalDB` o a `.\SQLEXPRESS` → Nueva consulta (en la base `master`) → pega el script → Ejecutar (F5).
- **Visual Studio:** Ver → Explorador de objetos de SQL Server → (localdb)\MSSQLLocalDB → clic derecho → Nueva consulta.
- **Consola:** `sqlcmd -S "(localdb)\MSSQLLocalDB" -i Ej01.sql`.
- Cada script elimina la base del ejercicio si ya existe y la crea de nuevo: así puedes repetir las pruebas desde cero.

3.4 Crear el proyecto

Archivo → Nuevo proyecto → **Aplicación de Windows Forms**. Agrega la referencia a SqlClient (NuGet si usas .NET moderno). Guarda la cadena de conexión en un único lugar (por ejemplo, una constante o App.config) para no repetirla en cada botón.

3.5 Convenciones usadas en la guía

Tema	Convención
Nombres de controles	Prefijos: txt (TextBox), cmb (ComboBox), dtp (DateTimePicker), btn (Button), lbl (Label), dgv (DataGridView).
Moneda	Pesos colombianos con punto de miles: $1.250.000. Los promedios se muestran redondeados a 2 decimales con coma: $1.104.166,67.
Mensajes	Los mensajes entre «comillas» deben aparecer exactamente así en un MessageBox o en un Label.
Fecha de referencia	Todos los casos dinámicos (año actual, mes actual, últimos N días/meses...) se calculan con la fecha de referencia **30/09/2026**. Si hoy es otro día, marca el `DateTimePicker dtpReferencia` (ShowCheckBox) con esa fecha para que tu consulta la use en lugar de GETDATE(); si no está marcado, tu consulta debe usar GETDATE().
Identificadores	Las columnas IDENTITY empiezan en 1 y crecen de a 1, en el orden de los INSERT.
Reglas de oro	1) Siempre parámetros SQL, nunca concatenar texto del usuario. 2) Siempre `using` para conexiones y comandos. 3) Nunca UPDATE/DELETE sin WHERE. 4) Valida antes de consultar. 5) Captura las excepciones y muestra mensajes claros.

3.6 Funciones de fecha de SQL Server (hoja de referencia)

Función	Qué hace	Ejemplo abstracto
GETDATE()	Fecha y hora actuales del servidor. Es el equivalente de `NOW()` de otros motores (**NOW() no existe en SQL Server**).	`SELECT GETDATE();`
CAST(... AS DATE)	Descarta la hora y deja solo la fecha.	`CAST(GETDATE() AS DATE)`
DATEADD(parte, n, fecha)	Suma (o resta, si n es negativo) n unidades: DAY, MONTH, YEAR...	`DATEADD(DAY, 5, '2026-01-10')` → 15/01/2026
DATEDIFF(parte, f1, f2)	Cuenta los límites de «parte» que se cruzan entre dos fechas (no son períodos completos).	`DATEDIFF(MONTH, '2026-01-31', '2026-02-01')` → 1
YEAR(), MONTH(), DAY()	Devuelven el año, mes o día como número.	`MONTH('2026-08-14')` → 8
DATEFROMPARTS(a, m, d)	Construye una fecha a partir de año, mes y día.	`DATEFROMPARTS(2026, 3, 1)`
EOMONTH(fecha [, n])	Último día del mes de la fecha (opcionalmente desplazado n meses).	`EOMONTH('2026-02-10')` → 28/02/2026

Otros motores	SQL Server
NOW()	GETDATE()
CURDATE()	CAST(GETDATE() AS DATE)
LIMIT n	TOP (n)
DATE_SUB / INTERVAL	DATEADD(parte, -n, fecha)
DATE_FORMAT	FORMAT() o CONVERT()

4. Nivel 1 — Ejercicios básicos

Conexión, lectura y las cuatro operaciones fundamentales. Cada ejercicio usa pocas tablas y datos pequeños (10 a 14 registros).

N.º	Ejercicio	Nivel	Casos de prueba
1	SELECT Y CONEXIÓN A SQL SERVER	Básico	8
2	INSERT — REGISTRAR PRODUCTOS	Básico	10
3	UPDATE — ACTUALIZAR CLIENTES	Básico	9
4	DELETE — ELIMINAR PROVEEDORES	Básico	7
5	CRUD COMPLETO DE EMPLEADOS	Básico	12

EJERCICIO 1 — SELECT Y CONEXIÓN A SQL SERVER

Nivel

Básico

Contexto

Una pequeña empresa guarda a sus empleados en SQL Server y necesita una ventana que muestre el listado completo para que Recursos Humanos pueda consultarlo sin abrir Management Studio.

Objetivo

Conectar una aplicación Windows Forms con una base local (LocalDB o SQL Express) usando **SqlConnection**, ejecutar un SELECT y mostrar el resultado en un **DataGridView** con **SqlDataAdapter** y **DataTable**. Practicar además el manejo básico de errores de conexión.

Base de datos

Nombre de la base de datos: `Ej01_Empleados`

Tabla `Empleados`

Columna	Tipo de datos	Restricciones
EmpleadoId	INT	PK, autoincremental
Nombre	NVARCHAR(50)	obligatorio
Apellido	NVARCHAR(50)	obligatorio
Cargo	NVARCHAR(40)	obligatorio
Salario	DECIMAL(12,2)	obligatorio
FechaIngreso	DATE	obligatorio

Relaciones: Una sola tabla, sin relaciones.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej01_Empleados') IS NOT NULL
BEGIN
    ALTER DATABASE Ej01_Empleados SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej01_Empleados;
END
GO

CREATE DATABASE Ej01_Empleados;
GO

USE Ej01_Empleados;
GO

CREATE TABLE Empleados (
    EmpleadoId    INT            IDENTITY(1,1) PRIMARY KEY,
    Nombre        NVARCHAR(50)   NOT NULL,
    Apellido      NVARCHAR(50)   NOT NULL,
    Cargo         NVARCHAR(40)   NOT NULL,
    Salario       DECIMAL(12,2)  NOT NULL,
    FechaIngreso  DATE           NOT NULL
);

INSERT INTO Empleados (Nombre, Apellido, Cargo, Salario, FechaIngreso) VALUES
    (N'Juan', N'Pérez', N'Vendedor', 2500000, '2022-03-15'),
    (N'María', N'Gómez', N'Contadora', 3800000, '2021-07-01'),
    (N'Carlos', N'Ramírez', N'Vendedor', 2600000, '2023-01-10'),
    (N'Ana', N'Torres', N'Analista', 3200000, '2020-11-23'),
    (N'Luis', N'Martínez', N'Bodeguero', 1900000, '2023-05-08'),
    (N'Sofía', N'López', N'Gerente', 7500000, '2019-02-14'),
```

```
    (N'Andrés', N'Castro', N'Vendedor', 2450000, '2024-02-19'),
    (N'Paula', N'Rojas', N'Analista', 3350000, '2022-09-05'),
    (N'Diego', N'Herrera', N'Bodeguero', 1950000, '2024-06-17'),
    (N'Laura', N'Mendoza', N'Recepcionista', 1800000, '2025-01-13'),
    (N'Felipe', N'Ortiz', N'Contador', 3900000, '2021-04-26'),
    (N'Camila', N'Vargas', N'Vendedora', 2550000, '2025-08-04');
```

Diseño del formulario

Control	Tipo	Uso
`btnCargar`	Button	Ejecuta la consulta y llena la grilla
`cmbOrden`	ComboBox	Opciones: Apellido / Salario (mayor a menor) / Fecha de ingreso (más reciente)
`dgvEmpleados`	DataGridView	Solo lectura, sin fila de inserción
`lblTotal`	Label	Muestra «Total de empleados: N»

Requerimientos funcionales

1. Al pulsar Cargar, el sistema debe abrir la conexión, consultar la tabla Empleados y mostrar todos los registros en el DataGridView.
2. El orden de los resultados debe depender de la opción elegida en cmbOrden (por defecto: Apellido).
3. lblTotal debe mostrar la cantidad de filas obtenidas.
4. La conexión debe abrirse y cerrarse correctamente en cada consulta (usa using).
5. Si no hay conexión con el servidor, se debe mostrar un mensaje claro y la grilla debe quedar vacía.
6. Si la tabla no tiene registros, se debe informar al usuario.
7. Pulsar Cargar varias veces no debe duplicar filas.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
SELECT	Listar todas las columnas de Empleados.
ORDER BY	Ordenar según la opción del ComboBox. El texto del ComboBox no debe concatenarse tal cual en la consulta.

Pistas (sin solución)

- Investiga la cadena de conexión de LocalDB: `(localdb)\MSSQLLocalDB`, y la de SQL Express: `.\SQLEXPRESS`.
- SqlDataAdapter.Fill(DataTable) abre y cierra la conexión por ti, pero practica también con SqlDataReader.
- Piensa cómo traducir cada opción del ComboBox a una cláusula ORDER BY fija (lista cerrada de opciones).
- Un DataTable nuevo en cada clic evita acumular filas.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Servidor inexistente o detenido	«No se pudo conectar con el servidor de base de datos.»
Tabla sin registros	«No hay empleados registrados.»
cmbOrden sin selección	Se usa Apellido por defecto (sin mensaje)

Casos de prueba

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Clic en Cargar	Orden por defecto (Apellido)	12 filas y 6 columnas (EmpleadoId, Nombre, Apellido, Cargo, Salario, FechaIngreso)
CP-02	Leer lblTotal	Después de CP-01	«Total de empleados: 12»
CP-03	Ordenar por Apellido	cmbOrden = Apellido	Primera fila: Andrés Castro · Última fila: Camila Vargas
CP-04	Ordenar por salario	cmbOrden = Salario (mayor a menor)	Primera fila: Sofía López $7.500.000 · Última fila: Laura Mendoza $1.800.000
CP-05	Ordenar por fecha	cmbOrden = Fecha de ingreso (más reciente)	Primera fila: Camila Vargas (04/08/2025) · Última fila: Sofía López (14/02/2019)
CP-06	Clic en Cargar dos veces seguidas	Sin modificar datos	Sigue habiendo 12 filas; lblTotal = «Total de empleados: 12»
CP-07	Conexión fallida	Cambiar el nombre del servidor por «.\NOEXISTE»	MessageBox «No se pudo conectar con el servidor de base de datos.»; grilla vacía
CP-08	Tabla vacía	Ejecutar `DELETE FROM Empleados;` y pulsar Cargar	0 filas; lblTotal = «Total de empleados: 0»; MessageBox «No hay empleados registrados.»

Reto adicional

Agrega un TextBox `txtSalarioMinimo`. Si contiene un valor, muestra solo los empleados con salario mayor o igual a ese valor. Con 3.000.000 deben aparecer 5 empleados; con 7.500.000, 1 empleado (Sofía López); con 8.000.000, 0 y un mensaje informativo.

Espacio para notas

EJERCICIO 2 — INSERT — REGISTRAR PRODUCTOS

Nivel

Básico

Contexto

Una tienda de tecnología recibe mercancía nueva cada semana. El encargado de bodega necesita un formulario para registrar productos nuevos y ver inmediatamente cómo queda el inventario.

Objetivo

Practicar INSERT con **SqlCommand** y parámetros (`Parameters.AddWithValue` o `Parameters.Add`), validar datos antes de enviarlos y reaccionar ante una violación de UNIQUE.

Base de datos

Nombre de la base de datos: `Ej02_Productos`

Tabla `Productos`

Columna	Tipo de datos	Restricciones
ProductoId	INT	PK, autoincremental
Codigo	NVARCHAR(10)	obligatorio, único
Nombre	NVARCHAR(80)	obligatorio
Precio	DECIMAL(12,2)	obligatorio, valida Precio > 0
Stock	INT	obligatorio, valida Stock >= 0

Relaciones: Una sola tabla. La columna Codigo es única.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej02_Productos') IS NOT NULL
BEGIN
    ALTER DATABASE Ej02_Productos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej02_Productos;
END
GO

CREATE DATABASE Ej02_Productos;
GO

USE Ej02_Productos;
GO

CREATE TABLE Productos (
    ProductoId  INT            IDENTITY(1,1) PRIMARY KEY,
    Codigo      NVARCHAR(10)   NOT NULL UNIQUE,
    Nombre      NVARCHAR(80)   NOT NULL,
    Precio      DECIMAL(12,2)  NOT NULL CHECK (Precio > 0),
    Stock       INT            NOT NULL CHECK (Stock >= 0)
);

INSERT INTO Productos (Codigo, Nombre, Precio, Stock) VALUES
    (N'P001', N'Mouse Óptico', 35000, 40),
    (N'P002', N'Teclado Básico', 62000, 25),
    (N'P003', N'Monitor 24 pulgadas', 690000, 8),
    (N'P004', N'Memoria USB 32GB', 28000, 60),
    (N'P005', N'Disco Duro 1TB', 260000, 15),
    (N'P006', N'Audífonos Bluetooth', 85000, 30),
    (N'P007', N'Cable HDMI', 18000, 100),
    (N'P008', N'Webcam HD', 120000, 12),
    (N'P009', N'Impresora Láser', 780000, 5),
    (N'P010', N'Router WiFi', 145000, 18);
```

Diseño del formulario

Control	Tipo	Uso
`txtCodigo`	TextBox	Código del producto (ej. P011)
`txtNombre`	TextBox	Nombre
`txtPrecio`	TextBox	Precio unitario
`txtStock`	TextBox	Unidades en inventario
`btnGuardar`	Button	Inserta el registro
`btnLimpiar`	Button	Limpia los campos
`dgvProductos`	DataGridView	Lista actualizada de productos
`lblTotal`	Label	Total de productos

Requerimientos funcionales

1. El usuario debe poder registrar un producto con código, nombre, precio y stock.
2. Todos los campos son obligatorios.
3. El precio debe ser un número mayor que cero.
4. El stock debe ser un entero mayor o igual a cero.
5. No se puede registrar un código que ya exista.
6. Después de guardar, la grilla debe refrescarse y mostrar el nuevo registro.
7. Después de guardar, los campos se limpian y el foco vuelve a txtCodigo.
8. Al abrir el formulario se deben mostrar los productos existentes.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
SELECT	Cargar los productos (ordenados por Codigo).
INSERT	Insertar un producto usando parámetros, nunca concatenando texto.

Pistas (sin solución)

- Convierte los TextBox con `decimal.TryParse` e `int.TryParse` antes de tocar la base.
- ExecuteNonQuery devuelve el número de filas afectadas.
- Investiga `SqlException.Number`: el error 2627 corresponde a violación de restricción UNIQUE.
- Decide si verificas el duplicado con un SELECT previo o capturando la excepción; prueba ambas.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Código vacío	«El código es obligatorio.»
Nombre vacío	«El nombre es obligatorio.»
Precio = abc	«El precio debe ser un número válido.»
Precio = -500 o 0	«El precio debe ser mayor que cero.»
Stock = 2,5	«El stock debe ser un número entero.»
Stock = -1	«El stock no puede ser negativo.»
Código repetido	«Ya existe un producto con el código P003.» (con el código ingresado)

Casos de prueba

Estado inicial: 10 productos. Ejecuta los casos en orden.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Cargar formulario	—	La grilla muestra 10 filas; lblTotal = «Total de productos: 10»
CP-02	Guardar producto válido	P011 · Teclado Mecánico · 185000 · 25	«Producto guardado correctamente.»; grilla con 11 filas; campos vacíos

ID	Acción	Datos de entrada	Resultado esperado
CP-03	Guardar con stock 0	P012 · Cable USB-C · 22000 · 0	«Producto guardado correctamente.»; 12 filas (stock 0 es válido)
CP-04	Nombre vacío	P013 · (vacío) · 50000 · 10	«El nombre es obligatorio.»; 12 filas (no se inserta)
CP-05	Código vacío	(vacío) · Parlante · 50000 · 10	«El código es obligatorio.»
CP-06	Precio no numérico	P013 · Parlante · abc · 10	«El precio debe ser un número válido.»
CP-07	Precio negativo	P013 · Parlante · -500 · 10	«El precio debe ser mayor que cero.»
CP-08	Stock decimal	P013 · Parlante · 50000 · 2,5	«El stock debe ser un número entero.»
CP-09	Stock negativo	P013 · Parlante · 50000 · -1	«El stock no puede ser negativo.»
CP-10	Código duplicado	P003 · Monitor 27 pulgadas · 850000 · 4	«Ya existe un producto con el código P003.»; 12 filas

Reto adicional

Genera automáticamente el siguiente código (P013, P014...) y muéstralo en txtCodigo de solo lectura al abrir el formulario. Con los datos del script, tras CP-03 el siguiente código debe ser P013.

Espacio para notas

EJERCICIO 3 — UPDATE — ACTUALIZAR CLIENTES

Nivel

Básico

Contexto

El área comercial cambia con frecuencia los datos de contacto de sus clientes. Se necesita una pantalla donde el asesor seleccione un cliente de la lista, corrija sus datos y guarde los cambios.

Objetivo

Practicar UPDATE con WHERE y parámetros, leer la fila seleccionada de un DataGridView, verificar `ExecuteNonQuery` (filas afectadas) y controlar duplicados.

Base de datos

Nombre de la base de datos: `Ej03_Clientes`

Tabla `Clientes`

Columna	Tipo de datos	Restricciones
ClienteId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
Telefono	NVARCHAR(15)	obligatorio
Email	NVARCHAR(100)	obligatorio, único
Ciudad	NVARCHAR(50)	obligatorio

Relaciones: Una sola tabla; Email es único.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej03_Clientes') IS NOT NULL
BEGIN
    ALTER DATABASE Ej03_Clientes SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej03_Clientes;
END
GO

CREATE DATABASE Ej03_Clientes;
GO

USE Ej03_Clientes;
GO

CREATE TABLE Clientes (
    ClienteId  INT            IDENTITY(1,1) PRIMARY KEY,
    Nombre     NVARCHAR(80)   NOT NULL,
    Telefono   NVARCHAR(15)   NOT NULL,
    Email      NVARCHAR(100)  NOT NULL UNIQUE,
    Ciudad     NVARCHAR(50)   NOT NULL
);

INSERT INTO Clientes (Nombre, Telefono, Email, Ciudad) VALUES
    (N'Carlos Andrade', N'3101234567', N'carlos.andrade@mail.com', N'Cali'),
    (N'Beatriz Salazar', N'3112345678', N'beatriz.salazar@mail.com', N'Bogotá'),
    (N'Jorge Cifuentes', N'3123456789', N'jorge.cifuentes@mail.com', N'Medellín'),
    (N'Carolina Mejía', N'3134567890', N'carolina.mejia@mail.com', N'Cali'),
    (N'Ricardo Bonilla', N'3145678901', N'ricardo.bonilla@mail.com', N'Barranquilla'),
    (N'Valentina Duarte', N'3156789012', N'valentina.duarte@mail.com', N'Bogotá'),
    (N'Héctor Londoño', N'3167890123', N'hector.londono@mail.com', N'Pereira'),
    (N'Natalia Quintero', N'3178901234', N'natalia.quintero@mail.com', N'Cali'),
    (N'Mauricio Ayala', N'3189012345', N'mauricio.ayala@mail.com', N'Bucaramanga'),
    (N'Daniela Ospina', N'3190123456', N'daniela.ospina@mail.com', N'Medellín'),
```

```
    (N'Sergio Palacios', N'3201234567', N'sergio.palacios@mail.com', N'Cartagena'),
    (N'Isabel Cárdenas', N'3212345678', N'isabel.cardenas@mail.com', N'Manizales');
```

Diseño del formulario

Control	Tipo	Uso
`dgvClientes`	DataGridView	Selección de fila completa, solo lectura
`txtId`	TextBox	Id (solo lectura)
`txtNombre`	TextBox	Nombre
`txtTelefono`	TextBox	Teléfono
`txtEmail`	TextBox	Correo
`cmbCiudad`	ComboBox	Ciudades (DropDownList)
`btnActualizar`	Button	Guarda los cambios
`lblTotal`	Label	Total de clientes

Requerimientos funcionales

1. Al iniciar, la grilla muestra todos los clientes.
2. Al seleccionar una fila, sus datos se copian a los controles de edición.
3. El usuario puede modificar nombre, teléfono, correo y ciudad.
4. El botón Actualizar modifica únicamente el cliente seleccionado (WHERE por ClienteId).
5. Si no hay fila seleccionada, se informa al usuario.
6. No se permiten correos repetidos en otro cliente.
7. Si el registro fue eliminado por otro usuario, se informa que ya no existe.
8. Tras actualizar, la grilla se refresca y conserva el registro visible.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
SELECT	Cargar la lista.
UPDATE	Actualizar cuatro columnas del cliente con ClienteId = @Id.

Pistas (sin solución)

- Un UPDATE sin WHERE modifica toda la tabla: comprueba el caso CP-01 mirando que solo cambie una fila.
- Compara el valor devuelto por ExecuteNonQuery con 0 y con 1.
- Valida el correo de forma sencilla (contiene «@» y un punto después); investiga `string.Contains` o `IndexOf`.
- El teléfono solo debe contener dígitos: recorre los caracteres con `char.IsDigit`.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Sin fila seleccionada	«Seleccione un cliente de la lista.»
Nombre vacío	«El nombre es obligatorio.»
Teléfono con letras	«El teléfono solo debe contener números.»
Correo sin @	«El correo electrónico no es válido.»
Correo usado por otro cliente	«Ya existe otro cliente con ese correo electrónico.»
Registro eliminado	«El cliente ya no existe en la base de datos.»

Casos de prueba

Estado inicial: 12 clientes; Medellín tiene 2 clientes (Id 3 y 10); Cali tiene 3 (Id 1, 4 y 8).

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Actualizar ciudad	Id 4 (Carolina Mejía): Ciudad Cali → Medellín	«Cliente actualizado correctamente.»; en SSMS solo cambia la fila 4; Medellín pasa a 3 clientes y Cali a 2

ID	Acción	Datos de entrada	Resultado esperado
CP-02	Actualizar correo	Id 10: Email → daniela.o@mail.com	«Cliente actualizado correctamente.»; la grilla muestra el nuevo correo
CP-03	Actualizar sin selección	Ninguna fila seleccionada	«Seleccione un cliente de la lista.»; sin cambios
CP-04	Nombre vacío	Id 2: Nombre = (vacío)	«El nombre es obligatorio.»
CP-05	Teléfono inválido	Id 2: Teléfono = 31A123	«El teléfono solo debe contener números.»
CP-06	Correo inválido	Id 2: Email = correo.com	«El correo electrónico no es válido.»
CP-07	Correo duplicado	Id 2: Email = carlos.andrade@mail.com (pertenece al Id 1)	«Ya existe otro cliente con ese correo electrónico.»; el Id 2 conserva su correo original
CP-08	Mismo correo propio	Id 2: cambiar solo Ciudad a Cali (Email sin cambios)	«Cliente actualizado correctamente.» (no debe marcar duplicado contra sí mismo)
CP-09	Registro inexistente	Seleccionar la fila Id 12, ejecutar `DELETE FROM Clientes WHERE ClienteId = 12;` en SSMS y pulsar Actualizar	«El cliente ya no existe en la base de datos.»

Reto adicional

Muestra en un Label cuántas filas fueron afectadas por la última operación y deshabilita btnActualizar mientras no haya cambios respecto a los datos originales de la fila seleccionada.

Espacio para notas

EJERCICIO 4 — DELETE — ELIMINAR PROVEEDORES

Nivel

Básico

Contexto

El departamento de compras depura su lista de proveedores. Algunos ya no trabajan con la empresa, pero otros tienen compras históricas que no deben perderse.

Objetivo

Practicar DELETE con WHERE, confirmación mediante **MessageBox** (Yes/No) y captura de la **SqlException** producida por una violación de llave foránea (error 547).

Base de datos

Nombre de la base de datos: `Ej04_Proveedores`

Tabla `Proveedores`

Columna	Tipo de datos	Restricciones
ProveedorId	INT	PK, autoincremental
Empresa	NVARCHAR(80)	obligatorio
Contacto	NVARCHAR(60)	obligatorio
Telefono	NVARCHAR(15)	obligatorio
Ciudad	NVARCHAR(50)	obligatorio
Estado	NVARCHAR(10)	obligatorio, por defecto 'Activo'

Tabla `Compras`

Columna	Tipo de datos	Restricciones
CompraId	INT	PK, autoincremental
ProveedorId	INT	FK → Proveedores(ProveedorId), obligatorio
Fecha	DATE	obligatorio
Valor	DECIMAL(14,2)	obligatorio

Relaciones: Compras.ProveedorId → Proveedores.ProveedorId (un proveedor puede tener varias compras). Los proveedores 1, 2 y 5 tienen compras registradas.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej04_Proveedores') IS NOT NULL
BEGIN
    ALTER DATABASE Ej04_Proveedores SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej04_Proveedores;
END
GO

CREATE DATABASE Ej04_Proveedores;
GO

USE Ej04_Proveedores;
GO

CREATE TABLE Proveedores (
    ProveedorId  INT           IDENTITY(1,1) PRIMARY KEY,
    Empresa      NVARCHAR(80)  NOT NULL,
    Contacto     NVARCHAR(60)  NOT NULL,
    Telefono     NVARCHAR(15)  NOT NULL,
```

```
    Ciudad       NVARCHAR(50)  NOT NULL,
    Estado       NVARCHAR(10)  NOT NULL DEFAULT 'Activo'
);

CREATE TABLE Compras (
    CompraId     INT           IDENTITY(1,1) PRIMARY KEY,
    ProveedorId  INT           NOT NULL REFERENCES Proveedores(ProveedorId),
    Fecha        DATE          NOT NULL,
    Valor        DECIMAL(14,2)  NOT NULL
);

INSERT INTO Proveedores (Empresa, Contacto, Telefono, Ciudad, Estado) VALUES
    (N'Distribuidora Andina', N'Marcela Ríos', N'6011234567', N'Bogotá', N'Activo'),
    (N'Tecnoimport SAS', N'Julián Prieto', N'6042345678', N'Medellín', N'Activo'),
    (N'Papelería del Norte', N'Gloria Benítez', N'6053456789', N'Barranquilla', N'Activo'),
    (N'Suministros Pacífico', N'Andrés Lozano', N'6024567890', N'Cali', N'Inactivo'),
    (N'Muebles Express', N'Paola Arenas', N'6015678901', N'Bogotá', N'Activo'),
    (N'Electro Sur', N'Fabián Gil', N'6026789012', N'Cali', N'Activo'),
    (N'Logística Express', N'Claudia Mora', N'6047890123', N'Medellín', N'Inactivo'),
    (N'Insumos del Café', N'Hernán Toro', N'6068901234', N'Manizales', N'Activo'),
    (N'Plásticos del Caribe', N'Sandra Polo', N'6059012345', N'Cartagena', N'Activo'),
    (N'Cartones Unidos', N'Iván Mejía', N'6070123456', N'Bucaramanga', N'Inactivo'),
    (N'Aseo Total', N'Liliana Paz', N'6011122233', N'Bogotá', N'Activo'),
    (N'Ferretería Central', N'Óscar Fonseca', N'6042233445', N'Medellín', N'Activo');

INSERT INTO Compras (ProveedorId, Fecha, Valor) VALUES
    (1, '2026-01-12', 4500000),
    (1, '2026-03-03', 2100000),
    (2, '2026-02-20', 8900000),
    (5, '2026-04-15', 3200000),
    (5, '2026-05-09', 1750000),
    (2, '2026-06-30', 5400000);
```

Diseño del formulario

Control	Tipo	Uso
`dgvProveedores`	DataGridView	Selección de fila completa
`btnEliminar`	Button	Elimina el proveedor seleccionado
`btnRefrescar`	Button	Recarga la lista
`lblTotal`	Label	Total de proveedores

Requerimientos funcionales

1. Al abrir el formulario se listan todos los proveedores.
2. El usuario selecciona una fila y pulsa Eliminar.
3. Antes de eliminar, el sistema pregunta «¿Desea eliminar al proveedor X?» con botones Sí/No.
4. Si el usuario responde No, no se modifica nada.
5. Si responde Sí, se ejecuta DELETE solo sobre ese ProveedorId.
6. Si el proveedor tiene compras, se informa que no puede eliminarse (sin cerrar la aplicación).
7. Si el registro ya no existe, se informa al usuario.
8. Después de eliminar, la grilla y lblTotal se actualizan.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
SELECT	Listar proveedores.
DELETE	Eliminar por ProveedorId con parámetro.

Pistas (sin solución)

- DialogResult y MessageBoxButtons.YesNo controlan la confirmación.
- Captura SqlException y revisa su propiedad Number (547 = conflicto con FOREIGN KEY).
- Toma el Id desde la fila seleccionada (`CurrentRow.Cells[...]`) y no desde el texto de la celda visible.
- Nunca pruebes DELETE sin WHERE en tu base real; la del ejercicio se reconstruye con el script.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Sin fila seleccionada	«Seleccione un proveedor para eliminar.»
Proveedor con compras	«No se puede eliminar el proveedor porque tiene compras registradas.»
Proveedor ya eliminado	«El proveedor ya no existe en la base de datos.»
Eliminación exitosa	«Proveedor eliminado correctamente.»

Casos de prueba

Estado inicial: 12 proveedores y 6 compras. Ejecuta los casos en orden.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Cancelar eliminación	Seleccionar Id 7 (Logística Express) → Sí/No: No	No cambia nada; 12 filas; lblTotal «Total de proveedores: 12»
CP-02	Eliminar correctamente	Seleccionar Id 7 → Sí	«Proveedor eliminado correctamente.»; 11 filas; el Id 7 ya no aparece
CP-03	Sin selección	Pulsar Eliminar sin fila seleccionada	«Seleccione un proveedor para eliminar.»
CP-04	Proveedor con compras	Seleccionar Id 5 (Muebles Express) → Sí	«No se puede eliminar el proveedor porque tiene compras registradas.»; siguen 11 filas
CP-05	Proveedor con compras (otro)	Seleccionar Id 1 → Sí	Mismo mensaje de CP-04; la tabla Compras conserva 6 filas
CP-06	Registro inexistente	Seleccionar Id 10, ejecutar `DELETE FROM Proveedores WHERE ProveedorId = 10;` en SSMS y pulsar Eliminar → Sí	«El proveedor ya no existe en la base de datos.»
CP-07	Eliminar sin compras	Después de CP-06, refrescar y eliminar el Id 4 → Sí	«Proveedor eliminado correctamente.»; quedan 9 filas

Reto adicional

Si el proveedor tiene compras, en lugar de bloquear la eliminación ofrece cambiar su Estado a «Inactivo» (UPDATE) con una segunda confirmación. Comprueba que el Id 5 queda «Inactivo» y conserva sus 2 compras.

Espacio para notas

EJERCICIO 5 — CRUD COMPLETO DE EMPLEADOS

Nivel

Básico

Contexto

Recursos Humanos necesita un único formulario para crear, consultar, modificar y eliminar empleados, sin depender del área de sistemas.

Objetivo

Integrar SELECT, INSERT, UPDATE y DELETE en una sola pantalla, con validaciones, confirmaciones y manejo de errores de base de datos.

Base de datos

Nombre de la base de datos: `Ej05_CrudEmpleados`

Tabla `Empleados`

Columna	Tipo de datos	Restricciones
EmpleadoId	INT	PK, autoincremental
Documento	NVARCHAR(15)	obligatorio, único
Nombre	NVARCHAR(50)	obligatorio
Apellido	NVARCHAR(50)	obligatorio
Cargo	NVARCHAR(40)	obligatorio
Salario	DECIMAL(12,2)	obligatorio, valida Salario > 0
FechaIngreso	DATE	obligatorio
Estado	NVARCHAR(10)	obligatorio

Relaciones: Una sola tabla; Documento es único.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej05_CrudEmpleados') IS NOT NULL
BEGIN
    ALTER DATABASE Ej05_CrudEmpleados SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej05_CrudEmpleados;
END
GO

CREATE DATABASE Ej05_CrudEmpleados;
GO

USE Ej05_CrudEmpleados;
GO

CREATE TABLE Empleados (
    EmpleadoId    INT            IDENTITY(1,1) PRIMARY KEY,
    Documento     NVARCHAR(15)   NOT NULL UNIQUE,
    Nombre        NVARCHAR(50)   NOT NULL,
    Apellido      NVARCHAR(50)   NOT NULL,
    Cargo         NVARCHAR(40)   NOT NULL,
    Salario       DECIMAL(12,2)  NOT NULL CHECK (Salario > 0),
    FechaIngreso  DATE           NOT NULL,
    Estado        NVARCHAR(10)   NOT NULL
);

INSERT INTO Empleados (Documento, Nombre, Apellido, Cargo, Salario, FechaIngreso, Estado) VALUES
    (N'1010101010', N'Laura', N'Giraldo', N'Vendedor', 2500000, '2022-02-01', N'Activo'),
    (N'1020202020', N'Mario', N'Bedoya', N'Contador', 3900000, '2021-06-15', N'Activo'),
```

```
(N'1030303030', N'Adriana', N'Nieto', N'Analista', 3300000, '2023-01-09', N'Activo'),
(N'1040404040', N'Camilo', N'Rueda', N'Bodeguero', 1900000, '2020-10-05', N'Inactivo'),
(N'1050505050', N'Patricia', N'Soto', N'Gerente', 7300000, '2018-03-19', N'Activo'),
(N'1060606060', N'Julián', N'Vera', N'Vendedor', 2450000, '2024-04-22', N'Activo'),
(N'1070707070', N'Sandra', N'Pineda', N'Analista', 3150000, '2022-08-30', N'Activo'),
(N'1080808080', N'Nicolás', N'Arias', N'Bodeguero', 1950000, '2023-11-13', N'Activo'),
(N'1090909090', N'Liliana', N'Cortés', N'Recepcionista', 1800000, '2025-02-03', N'Activo'),
(N'1101010101', N'Raúl', N'Medina', N'Vendedor', 2550000, '2021-09-27', N'Inactivo'),
(N'1111111111', N'Teresa', N'Uribe', N'Contador', 3850000, '2019-12-02', N'Activo'),
(N'1121212121', N'Felipe', N'Acosta', N'Analista', 3250000, '2024-07-08', N'Activo'),
(N'1131313131', N'Yolanda', N'Escobar', N'Vendedor', 2600000, '2025-05-19', N'Activo'),
(N'1141414141', N'Hernán', N'Guerra', N'Bodeguero', 1920000, '2022-12-12', N'Inactivo');
```

Diseño del formulario

Control	Tipo	Uso
`txtId`	TextBox	EmpleadoId (solo lectura)
`txtDocumento`	TextBox	Documento
`txtNombre`	TextBox	Nombre
`txtApellido`	TextBox	Apellido
`cmbCargo`	ComboBox	Vendedor, Contador, Analista, Bodeguero, Gerente, Recepcionista
`txtSalario`	TextBox	Salario
`dtpIngreso`	DateTimePicker	Formato corto
`cmbEstado`	ComboBox	Activo / Inactivo
`btnNuevo`	Button	Limpia el formulario
`btnGuardar`	Button	Inserta
`btnActualizar`	Button	Actualiza
`btnEliminar`	Button	Elimina
`dgvEmpleados`	DataGridView	Listado
`lblTotal`	Label	Total de empleados

Requerimientos funcionales

1. Al abrir, se listan todos los empleados.
2. El usuario puede registrar un empleado nuevo con todos los campos.
3. El documento no puede repetirse.
4. Nombre, apellido, documento, cargo y estado son obligatorios.
5. El salario debe ser un número mayor que cero.
6. La fecha de ingreso no puede ser posterior a hoy.
7. Al seleccionar una fila, sus datos se cargan en los controles.
8. El usuario puede actualizar el empleado seleccionado.
9. El usuario puede eliminar el empleado seleccionado, previa confirmación.
10. Tras cada operación, la grilla y lblTotal se actualizan.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
SELECT	Listar empleados.
INSERT	Registrar.
UPDATE	Modificar por EmpleadoId.
DELETE	Eliminar por EmpleadoId.

Pistas (sin solución)

- Reutiliza un método para validar y otro para recargar la grilla; evita copiar y pegar código.
- Habilita/deshabilita botones según haya o no una fila seleccionada.
- Para el duplicado puedes usar SELECT COUNT(*) previo o la SqlException 2627.
- `DateTimePicker.Value.Date` compara solo la fecha.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Documento duplicado	«Ya existe un empleado con el documento 1010101010.» (con el documento ingresado)
Nombre vacío	«El nombre es obligatorio.»
Salario = abc	«El salario debe ser un número válido.»
Salario = -500000	«El salario debe ser mayor que cero.»
Fecha futura	«La fecha de ingreso no puede ser futura.»
Estado sin elegir	«Seleccione un estado.»
Actualizar/eliminar sin selección	«Seleccione un empleado de la lista.»

Casos de prueba

Estado inicial: 14 empleados. Ejecuta los casos en orden; el conteo indicado es el esperado después de cada caso.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Cargar	—	14 filas; «Total de empleados: 14»
CP-02	Insertar válido	1088123456 · Tatiana · Rincón · Analista · 3100000 · 01/09/2026 · Activo	«Empleado guardado correctamente.»; 15 filas
CP-03	Documento duplicado	1010101010 · Pedro · Lara · Vendedor · 2000000 · 01/09/2026 · Activo	«Ya existe un empleado con el documento 1010101010.»; 15 filas
CP-04	Nombre vacío	1099000111 · (vacío) · Lara · Vendedor · 2000000 · Activo	«El nombre es obligatorio.»
CP-05	Salario negativo	1099000111 · Pedro · Lara · Vendedor · -500000	«El salario debe ser mayor que cero.»
CP-06	Salario texto	Salario = abc	«El salario debe ser un número válido.»
CP-07	Fecha futura	Fecha de ingreso = mañana	«La fecha de ingreso no puede ser futura.»
CP-08	Actualizar	Seleccionar documento 1010101010 (Laura Giraldo) y cambiar Salario a 2900000	«Empleado actualizado correctamente.»; la grilla muestra $2.900.000; 15 filas
CP-09	Actualizar sin selección	Pulsar Actualizar tras «Nuevo»	«Seleccione un empleado de la lista.»
CP-10	Cancelar eliminación	Seleccionar documento 1040404040 → No	15 filas; el registro sigue existiendo
CP-11	Eliminar	Seleccionar documento 1040404040 → Sí	«Empleado eliminado correctamente.»; 14 filas
CP-12	Eliminar inexistente	Seleccionar documento 1141414141, eliminarlo en SSMS y pulsar Eliminar → Sí	«El empleado ya no existe en la base de datos.»

Reto adicional

Agrega un ComboBox `cmbFiltroEstado` (Todos / Activo / Inactivo) que filtre la grilla. Con los datos iniciales: Activo = 11 empleados, Inactivo = 3.

Espacio para notas

5. Nivel 2 — Ejercicios intermedios

Búsquedas, filtros, agregados y rankings. Incluye tu primer ejercicio de fechas (rango entre dos DateTimePicker).

N.º	Ejercicio	Nivel	Casos de prueba
6	BÚSQUEDA POR NOMBRE (LIKE)	Intermedio	10
7	BÚSQUEDA POR APELLIDO Y OTROS CAMPOS	Intermedio	12
8	FILTROS POR RANGO DE FECHAS	Intermedio	10
9	FUNCIONES DE AGREGACIÓN: SUM, COUNT, AVG, MIN Y MAX	Intermedio	10
10	TOP Y ORDER BY: RANKINGS	Intermedio	10
11	FILTROS MÚLTIPLES COMBINADOS	Intermedio	15

EJERCICIO 6 — BÚSQUEDA POR NOMBRE (LIKE)

Nivel

Intermedio

Contexto

Una tienda tiene cientos de clientes y el cajero solo recuerda parte del nombre. Necesita escribir «Juan» y ver todos los clientes cuyo nombre contenga ese texto.

Objetivo

Dominar **LIKE** con comodines, parámetros SQL y el tratamiento de campos vacíos y búsquedas sin resultados.

Base de datos

Nombre de la base de datos: `Ej06_BusquedaNombre`

Tabla `Clientes`

Columna	Tipo de datos	Restricciones
ClienteId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
Ciudad	NVARCHAR(50)	obligatorio
Telefono	NVARCHAR(15)	obligatorio

Relaciones: Una sola tabla.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej06_BusquedaNombre') IS NOT NULL
BEGIN
    ALTER DATABASE Ej06_BusquedaNombre SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej06_BusquedaNombre;
END
GO

CREATE DATABASE Ej06_BusquedaNombre;
GO

USE Ej06_BusquedaNombre;
GO

CREATE TABLE Clientes (
    ClienteId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre     NVARCHAR(80)  NOT NULL,
    Ciudad     NVARCHAR(50)  NOT NULL,
    Telefono   NVARCHAR(15)  NOT NULL
);

INSERT INTO Clientes (Nombre, Ciudad, Telefono) VALUES
    (N'Juan Pérez', N'Bogotá', N'3110000000'),
    (N'Juana Ramírez', N'Cali', N'3111111111'),
    (N'Juan Carlos Gómez', N'Medellín', N'3112222222'),
    (N'María Fernanda López', N'Bogotá', N'3113333333'),
    (N'Mariana Torres', N'Cali', N'3114444444'),
    (N'Adriana Castro', N'Pereira', N'3115555555'),
    (N'Ana Sofía Rojas', N'Medellín', N'3116666666'),
    (N'Diana Ortiz', N'Bogotá', N'3117777777'),
    (N'Luis Juan Restrepo', N'Cali', N'3118888888'),
    (N'Pedro Salazar', N'Barranquilla', N'3119999999'),
    (N'Camilo Herrera', N'Bogotá', N'3121111110'),
    (N'Sebastián Cárdenas', N'Cartagena', N'3122222221'),
    (N'Paola Mendoza', N'Medellín', N'3123333332'),
```

```
    (N'Andrea Vargas', N'Cali', N'3124444443'),
    (N'Julián Ospina', N'Bogotá', N'3125555554');
```

Diseño del formulario

Control	Tipo	Uso
`txtNombre`	TextBox	Texto a buscar
`btnBuscar`	Button	Ejecuta la búsqueda
`btnMostrarTodos`	Button	Limpia el filtro y lista todo
`dgvClientes`	DataGridView	Resultados
`lblResultados`	Label	«Se encontraron N cliente(s)»

Requerimientos funcionales

1. El usuario escribe parte de un nombre y pulsa Buscar.
2. La búsqueda encuentra el texto en cualquier posición del nombre (coincidencia parcial).
3. No distingue mayúsculas de minúsculas.
4. Los espacios al inicio y al final del texto se ignoran.
5. Si el campo está vacío, se muestran todos los clientes.
6. Si no hay coincidencias, la grilla queda vacía y se muestra un mensaje.
7. lblResultados indica cuántos registros se encontraron.
8. Los caracteres especiales o comillas escritos por el usuario no deben romper la consulta.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
SELECT + WHERE	Filtrar por Nombre.
LIKE	Coincidencia parcial con comodín % construido desde el parámetro, no desde el texto SQL.
ORDER BY	Ordenar por Nombre.

Pistas (sin solución)

- El comodín % se une al valor del parámetro (en C# o en SQL), nunca se concatena el TextBox dentro del texto de la consulta.
- Investiga `string.Trim()`.
- Con la intercalación por defecto de SQL Server (CI) la búsqueda ya ignora mayúsculas; investiga qué pasa con las tildes (AS).
- Prueba el caso CP-08: si tu consulta devuelve filas, estás concatenando texto.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Campo vacío	Se muestran los 15 clientes (sin mensaje de error)
Sin coincidencias	«No se encontraron clientes con el nombre 'XYZ'.» (con el texto buscado)
Texto con espacios laterales	Se ignoran los espacios

Casos de prueba

Datos: 15 clientes.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Coincidencia exacta	Nombre: Juan Carlos Gómez	1 registro: Juan Carlos Gómez; «Se encontraron 1 cliente(s)»
CP-02	Coincidencia parcial	Nombre: Juan	4 registros: Juan Carlos Gómez, Juan Pérez, Juana Ramírez, Luis Juan Restrepo
CP-03	Minúsculas	Nombre: juan	Los mismos 4 registros del CP-02
CP-04	Parte intermedia	Nombre: ana	5 registros: Adriana Castro, Ana Sofía Rojas, Diana Ortiz, Juana Ramírez, Mariana Torres

ID	Acción	Datos de entrada	Resultado esperado
CP-05	Prefijo de otra palabra	Nombre: mar	2 registros: Mariana Torres, María Fernanda López
CP-06	Sin coincidencias	Nombre: XYZ	0 filas; «No se encontraron clientes con el nombre 'XYZ'.»
CP-07	Campo vacío	Nombre: (vacío)	15 filas; «Se encontraron 15 cliente(s)»
CP-08	Intento de inyección	Nombre: `Juan' OR '1'='1`	0 filas (se trata como texto); la aplicación no falla
CP-09	Espacios laterales	Nombre: « Pedro »	1 registro: Pedro Salazar
CP-10	Mostrar todos	Pulsar btnMostrarTodos tras CP-02	txtNombre vacío y 15 filas

Reto adicional

Haz que el texto «%» o «_» escrito por el usuario se busque literalmente (investiga ESCAPE en LIKE). Con «%» debe devolver 0 filas, no 15.

Espacio para notas

EJERCICIO 7 — BÚSQUEDA POR APELLIDO Y OTROS CAMPOS

Nivel

Intermedio

Contexto

La oficina de nómina busca empleados por distintos datos: a veces solo recuerdan el apellido, otras veces el cargo, la ciudad o el número de documento.

Objetivo

Construir una consulta cuyo campo de búsqueda cambia según lo elegido por el usuario, comparando **LIKE** (texto que comienza con…) contra igualdad exacta (**=**), sin abrir la puerta a inyección SQL.

Base de datos

Nombre de la base de datos: `Ej07_BusquedaApellido`

Tabla `Empleados`

Columna	Tipo de datos	Restricciones
EmpleadoId	INT	PK, autoincremental
Documento	NVARCHAR(15)	obligatorio, único
Nombre	NVARCHAR(50)	obligatorio
Apellido	NVARCHAR(50)	obligatorio
Cargo	NVARCHAR(40)	obligatorio
Ciudad	NVARCHAR(50)	obligatorio
Salario	DECIMAL(12,2)	obligatorio

Relaciones: Una sola tabla.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej07_BusquedaApellido') IS NOT NULL
BEGIN
    ALTER DATABASE Ej07_BusquedaApellido SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej07_BusquedaApellido;
END
GO

CREATE DATABASE Ej07_BusquedaApellido;
GO

USE Ej07_BusquedaApellido;
GO

CREATE TABLE Empleados (
    EmpleadoId  INT            IDENTITY(1,1) PRIMARY KEY,
    Documento   NVARCHAR(15)   NOT NULL UNIQUE,
    Nombre      NVARCHAR(50)   NOT NULL,
    Apellido    NVARCHAR(50)   NOT NULL,
    Cargo       NVARCHAR(40)   NOT NULL,
    Ciudad      NVARCHAR(50)   NOT NULL,
    Salario     DECIMAL(12,2)  NOT NULL
);

INSERT INTO Empleados (Documento, Nombre, Apellido, Cargo, Ciudad, Salario) VALUES
    (N'1010203040', N'Carlos', N'García', N'Vendedor', N'Cali', 2500000),
    (N'1020304050', N'Marta', N'Gómez', N'Contador', N'Bogotá', 3900000),
    (N'1030405060', N'Julián', N'García', N'Analista', N'Medellín', 3300000),
    (N'1040506070', N'Sara', N'Martínez', N'Vendedor', N'Cali', 2450000),
    (N'1050607080', N'Óscar', N'Garzón', N'Bodeguero', N'Bogotá', 1950000),
```

```
(N'1060708090', N'Lucía', N'Gómez', N'Vendedor', N'Medellín', 2550000),
(N'1070809010', N'Ricardo', N'Pardo', N'Gerente', N'Cali', 7200000),
(N'1080901020', N'Elena', N'García', N'Analista', N'Cali', 3250000),
(N'1090102030', N'Tomás', N'Martínez', N'Bodeguero', N'Barranquilla', 1900000),
(N'1101203040', N'Natalia', N'Suárez', N'Recepcionista', N'Bogotá', 1800000),
(N'1111304050', N'Hugo', N'Vélez', N'Vendedor', N'Barranquilla', 2400000),
(N'1121405060', N'Paola', N'Arango', N'Contador', N'Medellín', 3850000),
(N'1131506070', N'Iván', N'Correa', N'Analista', N'Bogotá', 3400000),
(N'1141607080', N'Daniela', N'Rincón', N'Vendedor', N'Bogotá', 2650000),
(N'1151708090', N'Mateo', N'Duque', N'Bodeguero', N'Cali', 1980000),
(N'1161809010', N'Valeria', N'Barrera', N'Gerente', N'Bogotá', 7600000),
(N'1171901020', N'Esteban', N'Zapata', N'Vendedor', N'Medellín', 2500000),
(N'1182002030', N'Camila', N'Peña', N'Analista', N'Barranquilla', 3150000),
(N'1192103040', N'Gabriel', N'Garcés', N'Contador', N'Cali', 3950000),
(N'1202204050', N'Juliana', N'Molina', N'Recepcionista', N'Medellín', 1850000);
```

Diseño del formulario

Control	Tipo	Uso
`cmbCampo`	ComboBox	Apellido / Cargo / Ciudad / Documento
`txtValor`	TextBox	Valor a buscar
`btnBuscar`	Button	Buscar
`dgvEmpleados`	DataGridView	Resultados
`lblResultados`	Label	Cantidad encontrada

Requerimientos funcionales

1. El usuario elige el campo y escribe el valor.
2. Apellido: coincidencia que COMIENZA con el texto (no cualquier posición).
3. Cargo, Ciudad y Documento: coincidencia exacta.
4. Si el valor está vacío, se muestra «Ingrese un valor de búsqueda.» y no se consulta.
5. Si no hay resultados, se informa el campo y el valor buscado.
6. El nombre de la columna debe salir de una lista cerrada en el código, nunca del texto libre.
7. Los resultados se ordenan por Apellido y Nombre.
8. lblResultados muestra «N empleado(s) encontrado(s)».

Consultas SQL que debe realizar

Tipo	Qué debes lograr
SELECT + WHERE	Filtro dinámico por columna.
LIKE	Prefijo para Apellido.
=	Igualdad exacta para los demás campos.
ORDER BY	Apellido, Nombre.

Pistas (sin solución)

- Una misma consulta no puede parametrizar el NOMBRE de una columna; decide cómo elegir el texto SQL a partir de una lista cerrada (switch).
- Compara el resultado de «Gar%» contra «%Gar%»: verás la diferencia entre prefijo y contiene (CP-03 vs reto).
- Documento es texto (NVARCHAR), no número.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
txtValor vacío	«Ingrese un valor de búsqueda.»
Sin resultados	«No se encontraron empleados con Documento = 999.» (campo y valor reales)
cmbCampo sin selección	«Seleccione el campo de búsqueda.»

Casos de prueba

Datos: 20 empleados.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Apellido exacto	Apellido: Gómez	2 registros: Marta Gómez, Lucía Gómez
CP-02	Apellido en minúscula	Apellido: gómez	Los mismos 2 registros
CP-03	Apellido por prefijo	Apellido: Gar	5 registros: Gabriel Garcés, Carlos García, Elena García, Julián García, Óscar Garzón
CP-04	Texto en medio del apellido	Apellido: arc	0 filas (la búsqueda es «comienza con»; ningún apellido empieza por «arc»)
CP-05	Por cargo	Cargo: Vendedor	6 registros; todos con Cargo = Vendedor
CP-06	Por ciudad	Ciudad: Cali	6 registros: Mateo Duque, Gabriel Garcés, Carlos García, Elena García, Sara Martínez, Ricardo Pardo
CP-07	Por documento existente	Documento: 1020304050	1 registro: Marta Gómez · Contador · Bogotá · $3.900.000
CP-08	Documento inexistente	Documento: 999	0 filas; «No se encontraron empleados con Documento = 999.»
CP-09	Documento parcial	Documento: 102030	0 filas (igualdad exacta)
CP-10	Cargo sin coincidencia	Cargo: Piloto	0 filas; mensaje de no encontrados
CP-11	Valor vacío	Cualquier campo, valor vacío	«Ingrese un valor de búsqueda.»; no se ejecuta consulta
CP-12	Cargo con pocos resultados	Cargo: Gerente	2 registros: Ricardo Pardo, Valeria Barrera

Reto adicional

Agrega la opción «Apellido (contiene)» a cmbCampo. Con «arc» debe devolver 4 registros y con «ar» debe devolver 10.

Espacio para notas

EJERCICIO 8 — FILTROS POR RANGO DE FECHAS

Nivel

Intermedio

Contexto

Contabilidad necesita listar los pedidos entre dos fechas elegidas en un calendario. Los pedidos guardan fecha y hora, por eso el último día del rango no debe perder pedidos de la tarde o la noche.

Objetivo

Usar dos **DateTimePicker** como parámetros de tipo fecha, comparar contra una columna DATETIME y entender por qué **BETWEEN** con una fecha «sin hora» puede dejar registros por fuera.

Base de datos

Nombre de la base de datos: `Ej08_FiltroFechas`

Tabla `Pedidos`

Columna	Tipo de datos	Restricciones
PedidoId	INT	PK, autoincremental
Cliente	NVARCHAR(80)	obligatorio
FechaPedido	DATETIME	obligatorio
Total	DECIMAL(14,2)	obligatorio

Relaciones: Una sola tabla.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej08_FiltroFechas') IS NOT NULL
BEGIN
    ALTER DATABASE Ej08_FiltroFechas SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej08_FiltroFechas;
END
GO

CREATE DATABASE Ej08_FiltroFechas;
GO

USE Ej08_FiltroFechas;
GO

CREATE TABLE Pedidos (
    PedidoId     INT            IDENTITY(1,1) PRIMARY KEY,
    Cliente      NVARCHAR(80)   NOT NULL,
    FechaPedido  DATETIME       NOT NULL,
    Total        DECIMAL(14,2)  NOT NULL
);

INSERT INTO Pedidos (Cliente, FechaPedido, Total) VALUES
    (N'Comercial Andina', '2026-01-15T09:30:00', 900000),
    (N'Distribuciones Sol', '2026-02-03T14:10:00', 1350000),
    (N'Papelería Central', '2026-02-28T23:50:00', 1400000),
    (N'Ferretería El Roble', '2026-03-01T00:00:00', 600000),
    (N'Supermercado La 14', '2026-03-01T08:15:00', 800000),
    (N'Tienda Mayorista', '2026-03-05T10:40:00', 2450000),
    (N'Comercial Andina', '2026-03-12T16:25:00', 300000),
    (N'Distribuciones Sol', '2026-03-15T12:00:00', 450000),
    (N'Papelería Central', '2026-03-20T18:45:00', 600000),
    (N'Ferretería El Roble', '2026-03-27T11:05:00', 950000),
    (N'Supermercado La 14', '2026-03-31T09:00:00', 2750000),
    (N'Tienda Mayorista', '2026-03-31T23:45:00', 1800000),
    (N'Comercial Andina', '2026-04-01T00:00:00', 850000),
```

```
(N'Distribuciones Sol', '2026-04-02T13:30:00', 1450000),
(N'Papelería Central', '2026-04-15T17:20:00', 2250000),
(N'Ferretería El Roble', '2026-05-06T10:10:00', 250000),
(N'Supermercado La 14', '2026-05-20T15:55:00', 1650000),
(N'Tienda Mayorista', '2026-06-01T08:00:00', 1750000),
(N'Comercial Andina', '2026-06-10T19:30:00', 1650000),
(N'Distribuciones Sol', '2026-06-30T23:59:00', 1400000),
(N'Papelería Central', '2026-07-04T12:45:00', 1750000),
(N'Ferretería El Roble', '2026-08-14T09:25:00', 2000000),
(N'Supermercado La 14', '2026-08-14T21:15:00', 800000),
(N'Tienda Mayorista', '2026-09-09T10:05:00', 2850000);
```

Diseño del formulario

Control	Tipo	Uso
`dtpInicio`	DateTimePicker	Fecha inicial (Format = Short)
`dtpFin`	DateTimePicker	Fecha final
`btnBuscar`	Button	Consultar
`dgvPedidos`	DataGridView	Pedidos del rango
`lblCantidad`	Label	Cantidad de pedidos
`lblTotal`	Label	Suma de totales

Requerimientos funcionales

1. El usuario elige fecha inicial y final y pulsa Buscar.
2. Se muestran los pedidos cuya fecha esté entre ambas fechas, incluyendo completo el día final (hasta las 23:59:59).
3. El día inicial se incluye desde las 00:00.
4. La fecha inicial no puede ser mayor que la final.
5. lblCantidad y lblTotal muestran el número de pedidos y la suma del rango.
6. Si no hay pedidos, se informa «No hay pedidos en el rango seleccionado.» y los totales quedan en 0.
7. Los resultados se ordenan por FechaPedido ascendente.
8. Las fechas se envían como parámetros de tipo fecha (SqlDbType.Date o DateTime), no como texto.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
SELECT + WHERE	Filtrar por rango de fechas.
BETWEEN / >= y <	Elige y justifica cuál usas con columnas DATETIME.
COUNT, SUM	Cantidad y total del rango.

Pistas (sin solución)

- Un DateTimePicker devuelve la hora actual del reloj en `Value`; usa `Value.Date`.
- Comparar `FechaPedido <= '2026-03-31'` equivale a las 00:00 de ese día. ¿Qué pedidos pierdes?
- Piensa en el rango semiabierto: fecha >= inicio AND fecha < fin + 1 día (`DATEADD(DAY, 1, …)`).
- El CP-02 (31/03 a las 23:45) es la prueba de que tu consulta es correcta.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Inicio > Fin	«La fecha inicial no puede ser mayor que la fecha final.»
Rango sin pedidos	«No hay pedidos en el rango seleccionado.»; Cantidad 0; Total $0

Casos de prueba

Datos: 24 pedidos (enero a septiembre 2026), varios con horas cercanas a la medianoche.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Mes completo	01/03/2026 a 31/03/2026	9 pedidos, suma $10.700.000 (PedidoId 4, 5, 6, 7, 8, 9, 10, 11, 12)

ID	Acción	Datos de entrada	Resultado esperado
CP-02	Último día con pedido nocturno	31/03/2026 a 31/03/2026	2 pedidos, suma $4.550.000: 09:00 y 23:45
CP-03	Día con dos pedidos	14/08/2026 a 14/08/2026	2 pedidos, suma $2.800.000
CP-04	Pedido a las 23:59	30/06/2026 a 30/06/2026	1 pedido, suma $1.400.000
CP-05	Cruce de mes (medianoche)	28/02/2026 a 01/03/2026	3 pedidos, suma $2.800.000 (incluye 28/02 23:50 y 01/03 00:00)
CP-06	Un trimestre	01/04/2026 a 30/06/2026	8 pedidos, suma $11.250.000
CP-07	Año completo	01/01/2026 a 31/12/2026	24 pedidos, suma $33.000.000
CP-08	Sin pedidos	01/01/2025 a 31/12/2025	0 pedidos, suma $0; «No hay pedidos en el rango seleccionado.»
CP-09	Fechas invertidas	Inicio 31/03/2026 · Fin 01/03/2026	«La fecha inicial no puede ser mayor que la fecha final.»; no se consulta
CP-10	Un solo pedido	01/09/2026 a 30/09/2026	1 pedido, suma $2.850.000

Reto adicional

Agrega un ComboBox «Atajo» con Hoy, Este mes y Mes pasado que ajuste los dos DateTimePicker. (Con fecha de sistema 30/09/2026: Este mes = 01/09–30/09 → 1 pedido; Mes pasado = 01/08–31/08 → 2 pedidos.)

Espacio para notas

EJERCICIO 9 — FUNCIONES DE AGREGACIÓN: SUM, COUNT, AVG, MIN Y MAX

Nivel

Intermedio

Contexto

La gerencia quiere ver, con un clic, cuántas ventas hay, cuánto suman, el promedio y la venta menor y mayor, según su estado. También necesita conocer el producto más caro y el más barato y el rango salarial de la empresa.

Objetivo

Usar funciones agregadas desde C# con **ExecuteScalar** o **SqlDataReader**, tratar valores NULL cuando no hay filas (SUM y AVG devuelven NULL) y combinar un agregado con una subconsulta o TOP para obtener el nombre del registro extremo.

Base de datos

Nombre de la base de datos: `Ej09_Agregados`

Tabla `Ventas`

Columna	Tipo de datos	Restricciones
VentaId	INT	PK, autoincremental
Vendedor	NVARCHAR(60)	obligatorio
Estado	NVARCHAR(12)	obligatorio
Total	DECIMAL(14,2)	obligatorio
Fecha	DATE	obligatorio

Tabla `Productos`

Columna	Tipo de datos	Restricciones
ProductoId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
Categoria	NVARCHAR(40)	obligatorio
Precio	DECIMAL(12,2)	obligatorio

Tabla `Empleados`

Columna	Tipo de datos	Restricciones
EmpleadoId	INT	PK, autoincremental
Nombre	NVARCHAR(60)	obligatorio
Cargo	NVARCHAR(40)	obligatorio
Salario	DECIMAL(12,2)	obligatorio

Relaciones: Tablas independientes (no hay relaciones).

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej09_Agregados') IS NOT NULL
BEGIN
    ALTER DATABASE Ej09_Agregados SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej09_Agregados;
END
GO
```

```
CREATE DATABASE Ej09_Agregados;
GO

USE Ej09_Agregados;
GO

CREATE TABLE Ventas (
    VentaId   INT            IDENTITY(1,1) PRIMARY KEY,
    Vendedor  NVARCHAR(60)   NOT NULL,
    Estado    NVARCHAR(12)   NOT NULL,
    Total     DECIMAL(14,2)  NOT NULL,
    Fecha     DATE           NOT NULL
);

CREATE TABLE Productos (
    ProductoId  INT            IDENTITY(1,1) PRIMARY KEY,
    Nombre      NVARCHAR(80)   NOT NULL,
    Categoria   NVARCHAR(40)   NOT NULL,
    Precio      DECIMAL(12,2)  NOT NULL
);

CREATE TABLE Empleados (
    EmpleadoId  INT            IDENTITY(1,1) PRIMARY KEY,
    Nombre      NVARCHAR(60)   NOT NULL,
    Cargo       NVARCHAR(40)   NOT NULL,
    Salario     DECIMAL(12,2)  NOT NULL
);

INSERT INTO Ventas (Vendedor, Estado, Total, Fecha) VALUES
    (N'Laura Vega', N'Pagada', 1650000, '2026-01-03'),
    (N'Mateo Ríos', N'Anulada', 800000, '2026-02-04'),
    (N'Carolina Paz', N'Pagada', 800000, '2026-03-05'),
    (N'Jorge Salas', N'Pagada', 1050000, '2026-04-06'),
    (N'Laura Vega', N'Pendiente', 450000, '2026-05-07'),
    (N'Mateo Ríos', N'Pagada', 650000, '2026-06-08'),
    (N'Carolina Paz', N'Pendiente', 700000, '2026-07-09'),
    (N'Jorge Salas', N'Anulada', 1900000, '2026-08-10'),
    (N'Laura Vega', N'Pagada', 2150000, '2026-09-11'),
    (N'Mateo Ríos', N'Pagada', 500000, '2026-01-12'),
    (N'Carolina Paz', N'Anulada', 1500000, '2026-02-13'),
    (N'Jorge Salas', N'Pendiente', 600000, '2026-03-14'),
    (N'Laura Vega', N'Pagada', 1200000, '2026-04-15'),
    (N'Mateo Ríos', N'Pagada', 950000, '2026-05-16'),
    (N'Carolina Paz', N'Pendiente', 2450000, '2026-06-17'),
    (N'Jorge Salas', N'Pagada', 1000000, '2026-07-18'),
    (N'Laura Vega', N'Pagada', 1600000, '2026-08-19'),
    (N'Mateo Ríos', N'Pagada', 550000, '2026-09-20'),
    (N'Carolina Paz', N'Pagada', 1150000, '2026-01-21'),
    (N'Jorge Salas', N'Pendiente', 950000, '2026-02-22');

INSERT INTO Productos (Nombre, Categoria, Precio) VALUES
    (N'Mouse Óptico', N'Periféricos', 35000),
    (N'Teclado Mecánico', N'Periféricos', 185000),
    (N'Monitor 24 pulgadas', N'Pantallas', 690000),
    (N'Monitor 32 pulgadas', N'Pantallas', 1290000),
    (N'Memoria USB 32GB', N'Almacenamiento', 28000),
    (N'Disco Duro 1TB', N'Almacenamiento', 260000),
    (N'Cable HDMI', N'Accesorios', 18000),
    (N'Webcam HD', N'Periféricos', 120000),
    (N'Impresora Láser', N'Impresión', 780000),
    (N'Router WiFi', N'Redes', 145000),
    (N'Audífonos Bluetooth', N'Accesorios', 85000),
    (N'Portátil 15 pulgadas', N'Computadores', 2890000);

INSERT INTO Empleados (Nombre, Cargo, Salario) VALUES
    (N'Juan Pérez', N'Vendedor', 2500000),
    (N'María Gómez', N'Contador', 3800000),
    (N'Carlos Ramírez', N'Vendedor', 2650000),
    (N'Ana Torres', N'Analista', 3200000),
    (N'Luis Martínez', N'Bodeguero', 1900000),
    (N'Sofía López', N'Gerente', 7500000),
    (N'Andrés Castro', N'Vendedor', 2400000),
    (N'Paula Rojas', N'Analista', 3350000),
    (N'Diego Herrera', N'Bodeguero', 1950000),
    (N'Laura Mendoza', N'Recepcionista', 1800000);
```

Diseño del formulario

Control	Tipo	Uso
`cmbEstado`	ComboBox	Todos / Pagada / Pendiente / Anulada / Rechazada
`btnCalcular`	Button	Estadísticas de ventas
`lblCantidad`	Label	COUNT
`lblTotal`	Label	SUM
`lblPromedio`	Label	AVG
`lblMinimo`	Label	MIN
`lblMaximo`	Label	MAX
`btnProductos`	Button	Producto más caro y más barato
`lblMasCaro`	Label	Nombre y precio
`lblMasBarato`	Label	Nombre y precio
`cmbCargo`	ComboBox	Todos / Vendedor / Analista / …
`btnSalarios`	Button	Salario mínimo y máximo
`lblSalMin`	Label	MIN salario
`lblSalMax`	Label	MAX salario

Requerimientos funcionales

1. btnCalcular muestra cantidad, total, promedio, mínimo y máximo de las ventas del estado elegido («Todos» no filtra).
2. Cuando no hay ventas, todos los valores se muestran como 0 (no como vacío ni error).
3. El promedio se muestra redondeado a 2 decimales.
4. Los valores monetarios se muestran con separador de miles y símbolo $.
5. btnProductos muestra nombre y precio del producto más caro y del más barato.
6. btnSalarios muestra salario mínimo y máximo, filtrando por cargo cuando no sea «Todos».
7. Un cargo sin empleados muestra 0 en ambos labels.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
COUNT	Cantidad de ventas.
SUM	Total vendido.
AVG	Promedio.
MIN / MAX	Venta menor y mayor; precio y salario extremos.
WHERE	Filtro por Estado o Cargo.
Subconsulta o TOP 1	Obtener el NOMBRE del producto más caro / más barato.

Pistas (sin solución)

- Investiga `ISNULL` o `COALESCE` para convertir NULL en 0.
- ExecuteScalar devuelve `DBNull.Value` cuando el agregado es NULL: no lo conviertas directamente.
- Hay tres formas de obtener el producto más caro: subconsulta con MAX, TOP 1 con ORDER BY DESC o un segundo SELECT. Prueba al menos dos.
- Para «Todos» puedes omitir el WHERE o usar un parámetro opcional.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Estado sin ventas (Rechazada)	Todos los labels en 0; sin errores
cmbEstado sin selección	«Seleccione un estado.»
Cargo sin empleados	Salarios mínimo y máximo = $0

Casos de prueba

Datos: 20 ventas (12 Pagadas, 5 Pendientes, 3 Anuladas), 12 productos y 10 empleados.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Estadísticas de todas las ventas	Estado: Todos	Cantidad 20 · Total $22.600.000 · Promedio $1.130.000 · Mínimo $450.000 · Máximo $2.450.000
CP-02	Solo pagadas	Estado: Pagada	Cantidad 12 · Total $13.250.000 · Promedio $1.104.166,67 · Mínimo $500.000 · Máximo $2.150.000
CP-03	Solo pendientes	Estado: Pendiente	Cantidad 5 · Total $5.150.000 · Promedio $1.030.000 · Mínimo $450.000 · Máximo $2.450.000
CP-04	Solo anuladas	Estado: Anulada	Cantidad 3 · Total $4.200.000 · Promedio $1.400.000 · Mínimo $800.000 · Máximo $1.900.000
CP-05	Estado sin registros	Estado: Rechazada	Cantidad 0 · Total $0 · Promedio $0 · Mínimo $0 · Máximo $0
CP-06	Producto más caro / más barato	Clic en btnProductos	Más caro: Portátil 15 pulgadas $2.890.000 · Más barato: Cable HDMI $18.000
CP-07	Salarios globales	Cargo: Todos	Mínimo $1.800.000 (Laura Mendoza) · Máximo $7.500.000 (Sofía López)
CP-08	Salarios de vendedores	Cargo: Vendedor	Mínimo $2.400.000 · Máximo $2.650.000
CP-09	Cargo inexistente	Cargo: Piloto	Mínimo $0 · Máximo $0
CP-10	Sin selección	cmbEstado vacío	«Seleccione un estado.»

Comprobación manual: ejecuta en SSMS `SELECT COUNT(*), SUM(Total), AVG(Total), MIN(Total), MAX(Total) FROM Ventas;` y verifica CP-01 (Total $22.600.000). Ventas de marzo 2026 (para el reto): 2 ventas, total $1.400.000.

Reto adicional

Agrega un segundo filtro por vendedor (cmbVendedor) que se combine con el estado. Con Estado = Pagada y Vendedor = «Laura Vega» debe dar: Cantidad 4 · Total $6.600.000 · Promedio $1.650.000 · Mínimo $1.200.000 · Máximo $2.150.000.

Espacio para notas

EJERCICIO 10 — TOP Y ORDER BY: RANKINGS

Nivel

Intermedio

Contexto

La dirección revisa rankings: productos más vendidos, mejores clientes, empleados mejor pagados y las ventas más grandes. Quiere escoger cuántos puestos ver.

Objetivo

Practicar **TOP (N)** con un valor parametrizado, **ORDER BY … DESC/ASC** y cómo elegir entre varias consultas según una opción del usuario.

Base de datos

Nombre de la base de datos: `Ej10_TopOrderBy`

Tabla `Productos`

Columna	Tipo de datos	Restricciones
ProductoId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
UnidadesVendidas	INT	obligatorio
Precio	DECIMAL(12,2)	obligatorio

Tabla `Clientes`

Columna	Tipo de datos	Restricciones
ClienteId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
NumCompras	INT	obligatorio

Tabla `Empleados`

Columna	Tipo de datos	Restricciones
EmpleadoId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
Salario	DECIMAL(12,2)	obligatorio

Tabla `Ventas`

Columna	Tipo de datos	Restricciones
VentaId	INT	PK, autoincremental
Descripcion	NVARCHAR(40)	obligatorio
Fecha	DATE	obligatorio
Total	DECIMAL(14,2)	obligatorio

Relaciones: Tablas independientes. Los valores de ordenamiento no tienen empates.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej10_TopOrderBy') IS NOT NULL
BEGIN
    ALTER DATABASE Ej10_TopOrderBy SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej10_TopOrderBy;
END
```

```
GO

CREATE DATABASE Ej10_TopOrderBy;
GO

USE Ej10_TopOrderBy;
GO

CREATE TABLE Productos (
    ProductoId        INT            IDENTITY(1,1) PRIMARY KEY,
    Nombre            NVARCHAR(80)   NOT NULL,
    UnidadesVendidas  INT            NOT NULL,
    Precio            DECIMAL(12,2)  NOT NULL
);

CREATE TABLE Clientes (
    ClienteId   INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre      NVARCHAR(80)  NOT NULL,
    NumCompras  INT           NOT NULL
);

CREATE TABLE Empleados (
    EmpleadoId  INT            IDENTITY(1,1) PRIMARY KEY,
    Nombre      NVARCHAR(80)   NOT NULL,
    Salario     DECIMAL(12,2)  NOT NULL
);

CREATE TABLE Ventas (
    VentaId      INT            IDENTITY(1,1) PRIMARY KEY,
    Descripcion  NVARCHAR(40)   NOT NULL,
    Fecha        DATE           NOT NULL,
    Total        DECIMAL(14,2)  NOT NULL
);

INSERT INTO Productos (Nombre, UnidadesVendidas, Precio) VALUES
    (N'Mouse Óptico', 307, 370000),
    (N'Teclado Mecánico', 31, 2300000),
    (N'Monitor 24"', 234, 860000),
    (N'Monitor 32"', 262, 1960000),
    (N'Memoria USB 32GB', 310, 2100000),
    (N'Disco Duro 1TB', 22, 1600000),
    (N'Cable HDMI', 120, 1490000),
    (N'Webcam HD', 251, 2480000),
    (N'Impresora Láser', 266, 1040000),
    (N'Router WiFi', 157, 1700000),
    (N'Audífonos Bluetooth', 349, 2000000),
    (N'Portátil 15"', 97, 830000),
    (N'Tablet 10"', 32, 1370000),
    (N'Parlante Portátil', 281, 2400000),
    (N'Cargador USB-C', 265, 2070000),
    (N'Silla Ergonómica', 182, 170000),
    (N'Escritorio Ajustable', 53, 1350000),
    (N'Lámpara LED', 142, 1140000),
    (N'Hub USB', 396, 2890000),
    (N'Soporte de Monitor', 199, 2020000);

INSERT INTO Clientes (Nombre, NumCompras) VALUES
    (N'Juan Pérez', 52),
    (N'Juana Ramírez', 18),
    (N'Carlos Gómez', 23),
    (N'María López', 45),
    (N'Mariana Torres', 38),
    (N'Adriana Castro', 31),
    (N'Ana Rojas', 30),
    (N'Diana Ortiz', 33),
    (N'Luis Restrepo', 7),
    (N'Pedro Salazar', 44),
    (N'Camilo Herrera', 40),
    (N'Sebastián Cárdenas', 57),
    (N'Paola Mendoza', 35),
    (N'Andrea Vargas', 13),
    (N'Julián Ospina', 17);

INSERT INTO Empleados (Nombre, Salario) VALUES
    (N'Mariana Torres', 7050000),
    (N'Adriana Castro', 4850000),
    (N'Ana Rojas', 2250000),
```

```
    (N'Diana Ortiz', 2200000),
    (N'Luis Restrepo', 8150000),
    (N'Pedro Salazar', 5650000),
    (N'Camilo Herrera', 2700000),
    (N'Sebastián Cárdenas', 2800000),
    (N'Paola Mendoza', 3700000),
    (N'Andrea Vargas', 6700000),
    (N'Julián Ospina', 6550000),
    (N'Juan Pérez', 3200000),
    (N'Juana Ramírez', 3000000),
    (N'Carlos Gómez', 7450000),
    (N'María López', 3900000);

INSERT INTO Ventas (Descripcion, Fecha, Total) VALUES
    (N'Venta 1', '2026-01-02', 5450000),
    (N'Venta 2', '2026-02-03', 1500000),
    (N'Venta 3', '2026-03-04', 2500000),
    (N'Venta 4', '2026-04-05', 3050000),
    (N'Venta 5', '2026-05-06', 2950000),
    (N'Venta 6', '2026-06-07', 3150000),
    (N'Venta 7', '2026-07-08', 1850000),
    (N'Venta 8', '2026-08-09', 4650000),
    (N'Venta 9', '2026-09-10', 2050000),
    (N'Venta 10', '2026-01-11', 1200000),
    (N'Venta 11', '2026-02-12', 4250000),
    (N'Venta 12', '2026-03-13', 3600000),
    (N'Venta 13', '2026-04-14', 1400000),
    (N'Venta 14', '2026-05-15', 5800000),
    (N'Venta 15', '2026-06-16', 1050000),
    (N'Venta 16', '2026-07-17', 2000000),
    (N'Venta 17', '2026-08-18', 3200000),
    (N'Venta 18', '2026-09-19', 2200000),
    (N'Venta 19', '2026-01-20', 1350000),
    (N'Venta 20', '2026-02-21', 4500000);
```

Diseño del formulario

Control	Tipo	Uso
`cmbRanking`	ComboBox	Productos más vendidos / Clientes con más compras / Empleados con mayor salario / Ventas más grandes / Productos más baratos
`txtCantidad`	TextBox	Cantidad de puestos (N)
`btnConsultar`	Button	Mostrar ranking
`dgvRanking`	DataGridView	Resultado
`lblInfo`	Label	Mensajes

Requerimientos funcionales

1. El usuario elige el ranking y escribe N.
2. La grilla muestra exactamente N filas ordenadas según el ranking elegido (o menos si la tabla tiene menos).
3. N debe ser un entero mayor que cero.
4. El ranking de productos muestra también el valor usado para ordenar.
5. «Productos más baratos» ordena ascendentemente por precio.
6. Si N supera el número de registros, se muestran todos.
7. Si N está vacío, se informa al usuario.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
TOP (N)	Limitar filas con N parametrizado.
ORDER BY DESC	Mayores primero.
ORDER BY ASC	Más baratos primero.

Pistas (sin solución)

- Desde SQL Server 2005, `TOP (@n)` acepta parámetros, pero requiere paréntesis.

- Sin ORDER BY, TOP devuelve filas arbitrarias.
- Cada opción del ComboBox puede mapearse a una consulta distinta dentro de un switch.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
N vacío	«Ingrese la cantidad de registros.»
N = 0 o negativo	«La cantidad debe ser mayor que cero.»
N = abc o 2,5	«La cantidad debe ser un número entero.»

Casos de prueba

Datos: 20 productos, 15 clientes, 15 empleados y 20 ventas (sin valores repetidos en las columnas de ranking).

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Top 5 productos más vendidos	N = 5	Hub USB (396), Audífonos Bluetooth (349), Memoria USB 32GB (310), Mouse Óptico (307), Parlante Portátil (281)
CP-02	Top 3 productos más vendidos	N = 3	Hub USB (396), Audífonos Bluetooth (349), Memoria USB 32GB (310)
CP-03	Top 5 clientes con más compras	N = 5	Sebastián Cárdenas (57), Juan Pérez (52), María López (45), Pedro Salazar (44), Camilo Herrera (40)
CP-04	Top 10 empleados mejor pagados	N = 10	Luis Restrepo ($8.150.000), Carlos Gómez ($7.450.000), Mariana Torres ($7.050.000), Andrea Vargas ($6.700.000), Julián Ospina ($6.550.000), Pedro Salazar ($5.650.000), Adriana Castro ($4.850.000), María López ($3.900.000), Paola Mendoza ($3.700.000), Juan Pérez ($3.200.000)
CP-05	Top 5 ventas más grandes	N = 5	Venta 14 ($5.800.000), Venta 1 ($5.450.000), Venta 8 ($4.650.000), Venta 20 ($4.500.000), Venta 11 ($4.250.000)
CP-06	5 productos más baratos	N = 5	Silla Ergonómica ($170.000), Mouse Óptico ($370.000), Portátil 15" ($830.000), Monitor 24" ($860.000), Impresora Láser ($1.040.000)
CP-07	N mayor que los registros	Productos más vendidos · N = 100	20 filas (todas), la primera es la del CP-01
CP-08	N = 0	N = 0	«La cantidad debe ser mayor que cero.»
CP-09	N texto	N = abc	«La cantidad debe ser un número entero.»
CP-10	N vacío	N = (vacío)	«Ingrese la cantidad de registros.»

Reto adicional

Agrega la opción «Clientes con menos compras». Con N = 3 deben salir: Luis Restrepo (7), Andrea Vargas (13), Julián Ospina (17).

Espacio para notas

EJERCICIO 11 — FILTROS MÚLTIPLES COMBINADOS

Nivel

Intermedio

Contexto

Recursos Humanos quiere un buscador avanzado: por nombre, cargo, estado, rango de salario y rango de fecha de ingreso, combinando solo los filtros que el usuario diligencie.

Objetivo

Construir dinámicamente una consulta con varias condiciones opcionales y sus parámetros, sin concatenar valores del usuario en el SQL.

Base de datos

Nombre de la base de datos: `Ej11_FiltrosMultiples`

Tabla `Empleados`

Columna	Tipo de datos	Restricciones
EmpleadoId	INT	PK, autoincremental
Nombre	NVARCHAR(50)	obligatorio
Apellido	NVARCHAR(50)	obligatorio
Cargo	NVARCHAR(40)	obligatorio
Estado	NVARCHAR(10)	obligatorio
Salario	DECIMAL(12,2)	obligatorio
FechaIngreso	DATE	obligatorio

Relaciones: Una sola tabla.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej11_FiltrosMultiples') IS NOT NULL
BEGIN
    ALTER DATABASE Ej11_FiltrosMultiples SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej11_FiltrosMultiples;
END
GO

CREATE DATABASE Ej11_FiltrosMultiples;
GO

USE Ej11_FiltrosMultiples;
GO

CREATE TABLE Empleados (
    EmpleadoId    INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre        NVARCHAR(50)  NOT NULL,
    Apellido      NVARCHAR(50)  NOT NULL,
    Cargo         NVARCHAR(40)  NOT NULL,
    Estado        NVARCHAR(10)  NOT NULL,
    Salario       DECIMAL(12,2) NOT NULL,
    FechaIngreso  DATE          NOT NULL
);

INSERT INTO Empleados (Nombre, Apellido, Cargo, Estado, Salario, FechaIngreso) VALUES
    (N'Andrés', N'Arias', N'Vendedor', N'Activo', 2400000, '2019-01-15'),
    (N'Beatriz', N'Henao', N'Analista', N'Activo', 3300000, '2019-04-22'),
    (N'Camilo', N'Benítez', N'Contador', N'Activo', 4000000, '2019-07-28'),
    (N'Daniela', N'Ibarra', N'Bodeguero', N'Inactivo', 2200000, '2019-11-02'),
    (N'Esteban', N'Cano', N'Gerente', N'Activo', 7400000, '2020-02-07'),
```

```
(N'Fabiola', N'Jaramillo', N'Vendedor', N'Activo', 2450000, '2020-05-14'),
(N'Gustavo', N'Dávila', N'Analista', N'Activo', 3350000, '2020-08-19'),
(N'Helena', N'Lara', N'Contador', N'Inactivo', 4050000, '2020-11-24'),
(N'Ignacio', N'Echeverri', N'Bodeguero', N'Activo', 2250000, '2021-03-01'),
(N'Jimena', N'Mejía', N'Gerente', N'Activo', 7000000, '2021-06-06'),
(N'Kevin', N'Franco', N'Vendedor', N'Activo', 2500000, '2021-09-11'),
(N'Lorena', N'Nieto', N'Analista', N'Inactivo', 3400000, '2021-12-17'),
(N'Mauricio', N'Giraldo', N'Contador', N'Activo', 4100000, '2022-03-24'),
(N'Natalia', N'Arias', N'Bodeguero', N'Activo', 2300000, '2022-06-29'),
(N'Óscar', N'Henao', N'Gerente', N'Activo', 7050000, '2022-10-04'),
(N'Patricia', N'Benítez', N'Vendedor', N'Inactivo', 2550000, '2023-01-09'),
(N'Ricardo', N'Ibarra', N'Analista', N'Activo', 3450000, '2023-04-16'),
(N'Sandra', N'Cano', N'Contador', N'Activo', 4150000, '2023-07-22'),
(N'Tomás', N'Jaramillo', N'Bodeguero', N'Activo', 1900000, '2023-10-27'),
(N'Úrsula', N'Dávila', N'Gerente', N'Inactivo', 7100000, '2024-02-01'),
(N'Valentina', N'Lara', N'Vendedor', N'Activo', 2600000, '2024-05-08'),
(N'Wilson', N'Echeverri', N'Analista', N'Activo', 3500000, '2024-08-13'),
(N'Ximena', N'Mejía', N'Contador', N'Activo', 4200000, '2024-11-18'),
(N'Yesid', N'Franco', N'Bodeguero', N'Inactivo', 1950000, '2025-02-23'),
(N'Zulma', N'Nieto', N'Gerente', N'Activo', 7150000, '2025-05-31');
```

Diseño del formulario

Control	Tipo	Uso
`txtNombre`	TextBox	Nombre (contiene)
`cmbCargo`	ComboBox	Todos + 5 cargos
`cmbEstado`	ComboBox	Todos / Activo / Inactivo
`txtSalarioMin`	TextBox	Salario mínimo (opcional)
`txtSalarioMax`	TextBox	Salario máximo (opcional)
`dtpDesde`	DateTimePicker	ShowCheckBox = true; solo se usa si está marcado
`dtpHasta`	DateTimePicker	ShowCheckBox = true
`btnBuscar`	Button	Aplicar filtros
`btnLimpiar`	Button	Restablecer filtros
`dgvEmpleados`	DataGridView	Resultados
`lblResultados`	Label	Cantidad

Requerimientos funcionales

1. Todos los filtros son opcionales; los vacíos no afectan la consulta.
2. Los filtros diligenciados se combinan con AND.
3. El nombre se busca como coincidencia parcial.
4. Cargo y Estado usan igualdad exacta (excepto «Todos»).
5. El salario admite mínimo, máximo o ambos (ambos extremos incluidos).
6. Las fechas de ingreso solo filtran si su casilla está marcada; ambos extremos se incluyen.
7. Se validan los rangos antes de consultar.
8. Si no hay resultados, se informa y la grilla queda vacía.
9. Limpiar restablece todos los controles y muestra todos los empleados.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
WHERE dinámico	Agregar condiciones según los filtros usados.
LIKE	Nombre.
=	Cargo y Estado.
>= / <=	Rangos de salario y de fecha.
ORDER BY	Apellido.

Pistas (sin solución)

- Empieza con `WHERE 1 = 1` y añade `AND …` por cada filtro activo (StringBuilder o string).

- Agrega el parámetro correspondiente solo cuando agregas su condición.
- Otra estrategia: parámetros opcionales del tipo (`@p IS NULL OR Columna = @p`). Compara ambas.
- `DateTimePicker.Checked` indica si la casilla está marcada.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Salario mínimo > máximo	«El salario mínimo no puede ser mayor que el máximo.»
Salario no numérico	«El salario mínimo debe ser un número válido.»
Salario negativo	«El salario máximo no puede ser negativo.»
Fecha inicial > final	«La fecha inicial no puede ser mayor que la fecha final.»
Sin resultados	«No se encontraron empleados con los filtros indicados.»

Casos de prueba

Datos: 25 empleados (6 inactivos). Los resultados de 4 o menos registros se listan por nombre.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Sin filtros	Todos los campos vacíos / «Todos»	25 registros
CP-02	Nombre contiene	Nombre: an	4 registros: Andrés Arias, Daniela Ibarra, Esteban Cano, Sandra Cano
CP-03	Nombre + cargo	Nombre: a · Cargo: Vendedor	4 registros: Andrés Arias, Fabiola Jaramillo, Patricia Benítez, Valentina Lara
CP-04	Solo estado	Estado: Inactivo	6 registros
CP-05	Cargo + estado	Cargo: Bodeguero · Estado: Inactivo	2 registros: Daniela Ibarra, Yesid Franco
CP-06	Rango de salario	Salario mín. 2.500.000 · máx. 3.500.000	8 registros
CP-07	Solo salario mínimo	Salario mín. 3.800.000	10 registros
CP-08	Rango de fechas	Ingreso entre 01/01/2021 y 31/12/2022	7 registros
CP-09	Cargo + estado + salario mín.	Analista · Activo · mín. 3.300.000	4 registros: Beatriz Henao, Gustavo Dávila, Ricardo Ibarra, Wilson Echeverri
CP-10	Combinación total	Nombre: a · Cargo: Vendedor · Estado: Activo · Salario 2.400.000–2.700.000 · Ingreso 2019–2023	2 registros: Andrés Arias, Fabiola Jaramillo
CP-11	Combinación sin resultados	Cargo: Gerente · Estado: Inactivo	1 registro: Úrsula Dávila
CP-12	Salario mínimo > máximo	mín. 4.000.000 · máx. 3.000.000	«El salario mínimo no puede ser mayor que el máximo.»
CP-13	Salario no numérico	Salario mín.: abc	«El salario mínimo debe ser un número válido.»
CP-14	Salario negativo	Salario máx.: -100	«El salario máximo no puede ser negativo.»
CP-15	Fecha inicial > final	Desde 01/01/2023 · Hasta 01/01/2022	«La fecha inicial no puede ser mayor que la fecha final.»

Reto adicional

Agrega un ComboBox «Ordenar por» (Salario, Fecha de ingreso, Apellido) y un resumen bajo la grilla con la suma y el promedio de salarios del resultado. Con el CP-06 la suma debe ser $24.650.000 y el promedio $3.081.250.

Espacio para notas

6. Nivel 3 — Intermedio/Avanzado y módulo de fechas

Relación entre tablas, agrupación y filtrado de grupos; después, el **módulo de fechas** (ejercicios 15 y 16), donde practicarás períodos fijos y relativos a la fecha actual: año y mes específicos, año y mes actual, anterior, últimos 7 y 30 días y últimos 10 y 12 meses.

N.º	Ejercicio	Nivel	Casos de prueba
12	INNER JOIN: CLIENTES Y VENTAS	Intermedio/Avanzado	9
13	GROUP BY: ESTADÍSTICAS POR GRUPO	Intermedio/Avanzado	9
14	HAVING: FILTRAR GRUPOS	Intermedio/Avanzado	10
15	CONSULTAS POR PERÍODOS DE TIEMPO	Intermedio/Avanzado	26
16	REPORTES Y ESTADÍSTICAS POR PERÍODO	Intermedio/Avanzado	11

EJERCICIO 12 — INNER JOIN: CLIENTES Y VENTAS

Nivel

Intermedio/Avanzado

Contexto

Una distribuidora guarda los clientes y las ventas en tablas separadas. El gerente necesita un listado que muestre el NOMBRE del cliente junto a cada venta, no su número de identificación.

Objetivo

Relacionar dos tablas con **INNER JOIN**, mostrar columnas de ambas en un DataGridView, filtrar por cliente y comprender qué clientes quedan por fuera de un INNER JOIN.

Base de datos

Nombre de la base de datos: `Ej12_InnerJoin`

Tabla `Clientes`

Columna	Tipo de datos	Restricciones
ClienteId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
Ciudad	NVARCHAR(50)	obligatorio

Tabla `Ventas`

Columna	Tipo de datos	Restricciones
VentaId	INT	PK, autoincremental
ClienteId	INT	FK → Clientes(ClienteId), obligatorio
Fecha	DATE	obligatorio
Total	DECIMAL(14,2)	obligatorio

Relaciones: Ventas.ClienteId → Clientes.ClienteId (un cliente tiene muchas ventas). Los clientes 9 y 10 no tienen ventas.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej12_InnerJoin') IS NOT NULL
BEGIN
    ALTER DATABASE Ej12_InnerJoin SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej12_InnerJoin;
END
GO

CREATE DATABASE Ej12_InnerJoin;
GO

USE Ej12_InnerJoin;
GO

CREATE TABLE Clientes (
    ClienteId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre     NVARCHAR(80)  NOT NULL,
    Ciudad     NVARCHAR(50)  NOT NULL
);

CREATE TABLE Ventas (
    VentaId    INT            IDENTITY(1,1) PRIMARY KEY,
    ClienteId  INT            NOT NULL REFERENCES Clientes(ClienteId),
    Fecha      DATE           NOT NULL,
    Total      DECIMAL(14,2)  NOT NULL
```

```
);

INSERT INTO Clientes (Nombre, Ciudad) VALUES
    (N'Carlos Andrade', N'Cali'),
    (N'Beatriz Salazar', N'Bogotá'),
    (N'Jorge Cifuentes', N'Medellín'),
    (N'Carolina Mejía', N'Cali'),
    (N'Ricardo Bonilla', N'Barranquilla'),
    (N'Valentina Duarte', N'Bogotá'),
    (N'Héctor Londoño', N'Pereira'),
    (N'Natalia Quintero', N'Cali'),
    (N'Mauricio Ayala', N'Bucaramanga'),
    (N'Daniela Ospina', N'Medellín');

INSERT INTO Ventas (ClienteId, Fecha, Total) VALUES
    (3, '2026-01-13', 800000),
    (5, '2026-01-18', 450000),
    (1, '2026-01-21', 2450000),
    (5, '2026-02-07', 1100000),
    (8, '2026-02-13', 350000),
    (6, '2026-02-25', 1600000),
    (1, '2026-03-17', 1100000),
    (8, '2026-04-22', 1800000),
    (8, '2026-05-04', 1350000),
    (3, '2026-05-08', 2150000),
    (4, '2026-05-12', 2250000),
    (8, '2026-06-12', 2550000),
    (6, '2026-06-17', 400000),
    (8, '2026-06-28', 1850000),
    (6, '2026-07-02', 1700000),
    (1, '2026-07-02', 1400000),
    (3, '2026-07-07', 900000),
    (7, '2026-07-18', 1300000),
    (6, '2026-07-22', 1150000),
    (5, '2026-07-22', 1700000),
    (3, '2026-08-16', 1750000),
    (6, '2026-09-01', 2450000),
    (4, '2026-09-06', 2950000),
    (2, '2026-09-06', 1750000),
    (7, '2026-09-08', 1000000);
```

Diseño del formulario

Control	Tipo	Uso
`cmbCliente`	ComboBox	«Todos» + los 10 clientes (DisplayMember = Nombre, ValueMember = ClienteId)
`dtpDesde`	DateTimePicker	ShowCheckBox = true
`dtpHasta`	DateTimePicker	ShowCheckBox = true
`btnConsultar`	Button	Consultar
`dgvVentas`	DataGridView	Cliente · Fecha · Total
`lblCantidad`	Label	Número de ventas
`lblTotal`	Label	Suma
`lblClientes`	Label	Clientes distintos

Requerimientos funcionales

1. El ComboBox se llena con una consulta a Clientes.
2. La grilla muestra exactamente tres columnas: Cliente, Fecha y Total (no muestra ids).
3. Sin filtros se muestran todas las ventas ordenadas por Fecha.
4. Si se elige un cliente, solo se muestran sus ventas.
5. Si las casillas de fecha están marcadas, se filtra por el rango (ambos extremos incluidos).
6. lblCantidad y lblTotal resumen lo que hay en la grilla.
7. lblClientes muestra cuántos clientes distintos aparecen en el resultado.
8. Si el cliente no tiene ventas, se informa con un mensaje.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
INNER JOIN	Ventas con Clientes por ClienteId.
WHERE	Cliente y rango de fechas opcionales.
ORDER BY	Fecha.
COUNT(DISTINCT)	Clientes distintos (investiga DISTINCT dentro de COUNT).

Pistas (sin solución)

- Escribe primero la consulta en SSMS y verifica que devuelva 25 filas antes de pasarla a C#.
- Usa alias de tabla (v, c) y nombra las columnas del resultado con AS.
- Para el ComboBox, asigna DataSource, DisplayMember y ValueMember; usa SelectedValue como parámetro.
- Al añadir «Todos» al ComboBox decide cómo representarás «sin filtro» (p. ej. ValueMember = 0).

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Fecha inicial > final	«La fecha inicial no puede ser mayor que la fecha final.»
Cliente sin ventas	«El cliente seleccionado no tiene ventas registradas.»
Sin resultados en el rango	«No se encontraron ventas con los filtros indicados.»

Casos de prueba

Datos: 10 clientes y 25 ventas en 2026.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Todas las ventas	Cliente: Todos · sin fechas	25 filas; suma $38.250.000; 3 columnas; lblClientes = 8
CP-02	Cliente con varias ventas	Cliente: Carlos Andrade	3 filas; suma $4.950.000
CP-03	Otro cliente	Cliente: Natalia Quintero	5 filas; suma $7.900.000
CP-04	Cliente sin ventas	Cliente: Mauricio Ayala	0 filas; «El cliente seleccionado no tiene ventas registradas.»
CP-05	Cliente sin ventas (2)	Cliente: Daniela Ospina	0 filas; mismo mensaje
CP-06	Rango de fechas	01/03/2026 a 30/06/2026 · Todos	8 filas; suma $13.450.000
CP-07	Cliente + rango	Jorge Cifuentes · 01/01/2026 a 30/06/2026	2 filas; suma $2.950.000
CP-08	Rango sin ventas	01/01/2025 a 31/12/2025	0 filas; «No se encontraron ventas con los filtros indicados.»
CP-09	Fechas invertidas	Desde 30/06/2026 · Hasta 01/03/2026	«La fecha inicial no puede ser mayor que la fecha final.»

Reto adicional

Sustituye INNER JOIN por LEFT JOIN y muestra también los clientes sin ventas (Fecha y Total vacíos). Con Cliente = Todos deben verse 27 filas (las 25 ventas + Mauricio Ayala + Daniela Ospina). Explica con tus palabras por qué INNER JOIN no las mostraba.

Espacio para notas

EJERCICIO 13 — GROUP BY: ESTADÍSTICAS POR GRUPO

Nivel

Intermedio/Avanzado

Contexto

La gerencia comercial quiere ver cuántas ventas, cuánto dinero y qué promedio genera cada vendedor, cada cliente, cada categoría, cada año y cada mes.

Objetivo

Agrupar con **GROUP BY** usando COUNT, SUM y AVG, combinar JOIN y agrupación, y agrupar por partes de una fecha con YEAR() y MONTH().

Base de datos

Nombre de la base de datos: `Ej13_GroupBy`

Tabla `Vendedores`

Columna	Tipo de datos	Restricciones
VendedorId	INT	PK, autoincremental
Nombre	NVARCHAR(60)	obligatorio

Tabla `Clientes`

Columna	Tipo de datos	Restricciones
ClienteId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio

Tabla `Categorias`

Columna	Tipo de datos	Restricciones
CategoriaId	INT	PK, autoincremental
Nombre	NVARCHAR(40)	obligatorio

Tabla `Ventas`

Columna	Tipo de datos	Restricciones
VentaId	INT	PK, autoincremental
VendedorId	INT	FK → Vendedores(VendedorId), obligatorio
ClienteId	INT	FK → Clientes(ClienteId), obligatorio
CategoriaId	INT	FK → Categorias(CategoriaId), obligatorio
FechaVenta	DATE	obligatorio
Total	DECIMAL(14,2)	obligatorio

Relaciones: Ventas.VendedorId → Vendedores; Ventas.ClienteId → Clientes; Ventas.CategoriaId → Categorias.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej13_GroupBy') IS NOT NULL
BEGIN
    ALTER DATABASE Ej13_GroupBy SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej13_GroupBy;
END
GO
```

```
CREATE DATABASE Ej13_GroupBy;
GO

USE Ej13_GroupBy;
GO

CREATE TABLE Vendedores (
    VendedorId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre      NVARCHAR(60)  NOT NULL
);

CREATE TABLE Clientes (
    ClienteId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre     NVARCHAR(80)  NOT NULL
);

CREATE TABLE Categorias (
    CategoriaId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre       NVARCHAR(40)  NOT NULL
);

CREATE TABLE Ventas (
    VentaId      INT            IDENTITY(1,1) PRIMARY KEY,
    VendedorId   INT            NOT NULL REFERENCES Vendedores(VendedorId),
    ClienteId    INT            NOT NULL REFERENCES Clientes(ClienteId),
    CategoriaId  INT            NOT NULL REFERENCES Categorias(CategoriaId),
    FechaVenta   DATE           NOT NULL,
    Total        DECIMAL(14,2)  NOT NULL
);

INSERT INTO Vendedores (Nombre) VALUES
    (N'Laura Vega'),
    (N'Mateo Ríos'),
    (N'Carolina Paz'),
    (N'Jorge Salas'),
    (N'Diana Mora');

INSERT INTO Clientes (Nombre) VALUES
    (N'Comercial Andina'),
    (N'Distribuciones Sol'),
    (N'Papelería Central'),
    (N'Ferretería El Roble'),
    (N'Supermercado La 14'),
    (N'Tienda Mayorista'),
    (N'Ofimática Cali'),
    (N'Servicios Delta');

INSERT INTO Categorias (Nombre) VALUES
    (N'Tecnología'),
    (N'Hogar'),
    (N'Oficina'),
    (N'Deportes');

INSERT INTO Ventas (VendedorId, ClienteId, CategoriaId, FechaVenta, Total) VALUES
    (2, 1, 2, '2024-03-16', 650000),
    (1, 3, 1, '2024-06-28', 1600000),
    (1, 5, 1, '2024-08-27', 1700000),
    (1, 3, 2, '2024-09-07', 1150000),
    (2, 1, 1, '2024-10-16', 1750000),
    (3, 3, 3, '2024-10-17', 650000),
    (3, 6, 4, '2024-11-12', 400000),
    (2, 7, 1, '2024-12-06', 1800000),
    (2, 4, 2, '2025-01-03', 950000),
    (3, 1, 1, '2025-05-06', 1000000),
    (4, 5, 1, '2025-05-11', 1150000),
    (3, 8, 2, '2025-05-13', 1200000),
    (4, 7, 2, '2025-05-15', 1000000),
    (4, 6, 1, '2025-06-17', 1100000),
    (3, 1, 3, '2025-06-25', 500000),
    (2, 5, 4, '2025-06-27', 550000),
    (4, 8, 3, '2025-07-05', 300000),
    (1, 3, 3, '2025-08-06', 850000),
    (2, 6, 3, '2025-08-06', 350000),
    (1, 6, 3, '2025-08-16', 500000),
    (1, 5, 4, '2025-08-17', 600000),
    (3, 2, 1, '2025-11-05', 1350000),
    (5, 6, 3, '2025-11-24', 600000),
```

```
(1, 5, 3, '2025-11-24', 300000),
(3, 8, 3, '2026-01-11', 450000),
(5, 1, 1, '2026-03-05', 1050000),
(1, 2, 2, '2026-03-13', 1150000),
(1, 3, 2, '2026-03-21', 1200000),
(3, 4, 2, '2026-03-25', 1000000),
(4, 3, 1, '2026-04-19', 1600000),
(2, 6, 1, '2026-04-24', 1450000),
(5, 3, 2, '2026-05-15', 1200000),
(2, 6, 1, '2026-05-22', 1300000),
(5, 3, 1, '2026-06-02', 1550000),
(1, 8, 4, '2026-06-10', 250000),
(2, 2, 2, '2026-07-06', 800000),
(1, 6, 2, '2026-09-03', 1100000),
(4, 7, 1, '2026-09-10', 1750000),
(5, 5, 4, '2026-09-13', 200000),
(1, 3, 2, '2026-09-21', 1200000);
```

Diseño del formulario

Control	Tipo	Uso
`cmbAgrupar`	ComboBox	Vendedor / Cliente / Categoría / Año / Mes
`txtAnio`	TextBox	Solo para «Mes»: año a analizar
`btnGenerar`	Button	Generar reporte
`dgvReporte`	DataGridView	Grupo · Cantidad · Total · Promedio
`lblTotalGeneral`	Label	Total de todos los grupos

Requerimientos funcionales

1. El usuario elige la forma de agrupar y pulsa Generar.
2. Vendedor, Cliente y Categoría muestran el NOMBRE (no el id), ordenados por Total de mayor a menor.
3. Año muestra un renglón por año, ordenado cronológicamente.
4. Mes muestra un renglón por mes del año indicado (solo meses con ventas), con el nombre del mes en español, en orden de calendario.
5. Cada grupo muestra Cantidad, Total y Promedio (promedio redondeado a 2 decimales).
6. txtAnio solo se habilita y valida cuando se elige «Mes».
7. lblTotalGeneral muestra la suma de la columna Total de la grilla.
8. Si el año no tiene ventas se informa y la grilla queda vacía.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
GROUP BY	Cinco agrupaciones distintas.
COUNT, SUM, AVG	Por grupo.
INNER JOIN	Para mostrar nombres.
YEAR(), MONTH()	Agrupar por partes de la fecha.
ORDER BY	Total DESC o fecha ASC según la agrupación.

Pistas (sin solución)

- Toda columna del SELECT que no sea agregada debe aparecer en el GROUP BY.
- Para agrupar por mes sin mezclar años, filtra primero el año con WHERE (no con HAVING).
- Convierte el número de mes en nombre con un arreglo en C# o con CASE en SQL; decide cuál prefieres y por qué.
- Compara la suma de la columna Total de la grilla con `SELECT SUM(Total) FROM Ventas`.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Año vacío (modo Mes)	«Ingrese el año.»
Año = abc	«El año debe ser un número válido.»
Año = 1800	«El año debe estar entre 2000 y 2100.»
Año sin ventas	«No hay ventas registradas en el año 2030.»

Casos de prueba

Datos: 5 vendedores, 8 clientes, 4 categorías y 40 ventas (8 de 2024, 16 de 2025, 16 de 2026). Total de todas las ventas: $39.250.000.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Agrupar por vendedor	cmbAgrupar = Vendedor	5 filas; total general $39.250.000. Ver Tabla 13-A
CP-02	Agrupar por cliente	cmbAgrupar = Cliente	8 filas. Ver Tabla 13-B
CP-03	Agrupar por categoría	cmbAgrupar = Categoría	4 filas. Ver Tabla 13-C
CP-04	Agrupar por año	cmbAgrupar = Año	3 filas (2024, 2025, 2026). Ver Tabla 13-D
CP-05	Agrupar por mes	cmbAgrupar = Mes · Año 2026	7 filas. Ver Tabla 13-E
CP-06	Agrupar por mes	cmbAgrupar = Mes · Año 2025	6 filas. Ver Tabla 13-F
CP-07	Año sin ventas	Mes · Año 2030	0 filas; «No hay ventas registradas en el año 2030.»
CP-08	Año vacío	Mes · Año (vacío)	«Ingrese el año.»
CP-09	Año inválido	Mes · Año abc	«El año debe ser un número válido.»

Tabla 13-A — Por vendedor (orden Total DESC)

Grupo	Cantidad	Total	Promedio
Laura Vega	12	$11.600.000	$966.666,67
Mateo Ríos	9	$9.600.000	$1.066.666,67
Jorge Salas	6	$6.900.000	$1.150.000
Carolina Paz	8	$6.550.000	$818.750
Diana Mora	5	$4.600.000	$920.000

Tabla 13-B — Por cliente (orden Total DESC)

Grupo	Cantidad	Total	Promedio
Papelería Central	9	$11.000.000	$1.222.222,22
Tienda Mayorista	8	$6.800.000	$850.000
Comercial Andina	5	$4.950.000	$990.000
Ofimática Cali	3	$4.550.000	$1.516.666,67
Supermercado La 14	6	$4.500.000	$750.000
Distribuciones Sol	3	$3.300.000	$1.100.000
Servicios Delta	4	$2.200.000	$550.000
Ferretería El Roble	2	$1.950.000	$975.000

Tabla 13-C — Por categoría (orden Total DESC)

Grupo	Cantidad	Total	Promedio
Tecnología	14	$20.150.000	$1.439.285,71
Hogar	12	$12.600.000	$1.050.000
Oficina	9	$4.500.000	$500.000
Deportes	5	$2.000.000	$400.000

Tabla 13-D — Por año

Año	Cantidad	Total	Promedio
2024	8	$9.700.000	$1.212.500
2025	16	$12.300.000	$768.750
2026	16	$17.250.000	$1.078.125

Tabla 13-E — Por mes, año 2026

Mes	Cantidad	Total	Promedio
Enero	1	$450.000	$450.000
Marzo	4	$4.400.000	$1.100.000
Abril	2	$3.050.000	$1.525.000
Mayo	2	$2.500.000	$1.250.000
Junio	2	$1.800.000	$900.000
Julio	1	$800.000	$800.000
Septiembre	4	$4.250.000	$1.062.500

Tabla 13-F — Por mes, año 2025

Mes	Cantidad	Total	Promedio
Enero	1	$950.000	$950.000
Mayo	4	$4.350.000	$1.087.500
Junio	3	$2.150.000	$716.666,67
Julio	1	$300.000	$300.000
Agosto	4	$2.300.000	$575.000
Noviembre	3	$2.250.000	$750.000

Reto adicional

Agrega una agrupación doble «Vendedor y Categoría» (8 o más filas) y una fila final con el total general usando ROLLUP o una segunda consulta. El total general debe ser $39.250.000.

Espacio para notas

EJERCICIO 14 — HAVING: FILTRAR GRUPOS

Nivel

Intermedio/Avanzado

Contexto

Dirección comercial solo quiere ver los grupos «importantes»: vendedores con más de cierto número de ventas, categorías cuyo total supere un valor, clientes cuyo ticket promedio sea alto.

Objetivo

Entender la diferencia entre **WHERE** (filtra filas antes de agrupar) y **HAVING** (filtra grupos después de agrupar), usando agregados en la condición.

Base de datos

Nombre de la base de datos: `Ej14_Having`

Tabla `Vendedores`

Columna	Tipo de datos	Restricciones
VendedorId	INT	PK, autoincremental
Nombre	NVARCHAR(60)	obligatorio

Tabla `Clientes`

Columna	Tipo de datos	Restricciones
ClienteId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio

Tabla `Categorias`

Columna	Tipo de datos	Restricciones
CategoriaId	INT	PK, autoincremental
Nombre	NVARCHAR(40)	obligatorio

Tabla `Ventas`

Columna	Tipo de datos	Restricciones
VentaId	INT	PK, autoincremental
VendedorId	INT	FK → Vendedores(VendedorId), obligatorio
ClienteId	INT	FK → Clientes(ClienteId), obligatorio
CategoriaId	INT	FK → Categorias(CategoriaId), obligatorio
FechaVenta	DATE	obligatorio
Total	DECIMAL(14,2)	obligatorio

Relaciones: Las mismas del Ejercicio 13 (datos idénticos).

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej14_Having') IS NOT NULL
BEGIN
    ALTER DATABASE Ej14_Having SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej14_Having;
END
GO

CREATE DATABASE Ej14_Having;
```

```
GO

USE Ej14_Having;
GO

CREATE TABLE Vendedores (
    VendedorId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre      NVARCHAR(60)  NOT NULL
);

CREATE TABLE Clientes (
    ClienteId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre     NVARCHAR(80)  NOT NULL
);

CREATE TABLE Categorias (
    CategoriaId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre       NVARCHAR(40)  NOT NULL
);

CREATE TABLE Ventas (
    VentaId      INT            IDENTITY(1,1) PRIMARY KEY,
    VendedorId   INT            NOT NULL REFERENCES Vendedores(VendedorId),
    ClienteId    INT            NOT NULL REFERENCES Clientes(ClienteId),
    CategoriaId  INT            NOT NULL REFERENCES Categorias(CategoriaId),
    FechaVenta   DATE           NOT NULL,
    Total        DECIMAL(14,2)  NOT NULL
);

INSERT INTO Vendedores (Nombre) VALUES
    (N'Laura Vega'),
    (N'Mateo Ríos'),
    (N'Carolina Paz'),
    (N'Jorge Salas'),
    (N'Diana Mora');

INSERT INTO Clientes (Nombre) VALUES
    (N'Comercial Andina'),
    (N'Distribuciones Sol'),
    (N'Papelería Central'),
    (N'Ferretería El Roble'),
    (N'Supermercado La 14'),
    (N'Tienda Mayorista'),
    (N'Ofimática Cali'),
    (N'Servicios Delta');

INSERT INTO Categorias (Nombre) VALUES
    (N'Tecnología'),
    (N'Hogar'),
    (N'Oficina'),
    (N'Deportes');

INSERT INTO Ventas (VendedorId, ClienteId, CategoriaId, FechaVenta, Total) VALUES
    (2, 1, 2, '2024-03-16', 650000),
    (1, 3, 1, '2024-06-28', 1600000),
    (1, 5, 1, '2024-08-27', 1700000),
    (1, 3, 2, '2024-09-07', 1150000),
    (2, 1, 1, '2024-10-16', 1750000),
    (3, 3, 3, '2024-10-17', 650000),
    (3, 6, 4, '2024-11-12', 400000),
    (2, 7, 1, '2024-12-06', 1800000),
    (2, 4, 2, '2025-01-03', 950000),
    (3, 1, 1, '2025-05-06', 1000000),
    (4, 5, 1, '2025-05-11', 1150000),
    (3, 8, 2, '2025-05-13', 1200000),
    (4, 7, 2, '2025-05-15', 1000000),
    (4, 6, 1, '2025-06-17', 1100000),
    (3, 1, 3, '2025-06-25', 500000),
    (2, 5, 4, '2025-06-27', 550000),
    (4, 8, 3, '2025-07-05', 300000),
    (1, 3, 3, '2025-08-06', 850000),
    (2, 6, 3, '2025-08-06', 350000),
    (1, 6, 3, '2025-08-16', 500000),
    (1, 5, 4, '2025-08-17', 600000),
    (3, 2, 1, '2025-11-05', 1350000),
    (5, 6, 3, '2025-11-24', 600000),
    (1, 5, 3, '2025-11-24', 300000),
```

```
(3, 8, 3, '2026-01-11', 450000),
(5, 1, 1, '2026-03-05', 1050000),
(1, 2, 2, '2026-03-13', 1150000),
(1, 3, 2, '2026-03-21', 1200000),
(3, 4, 2, '2026-03-25', 1000000),
(4, 3, 1, '2026-04-19', 1600000),
(2, 6, 1, '2026-04-24', 1450000),
(5, 3, 2, '2026-05-15', 1200000),
(2, 6, 1, '2026-05-22', 1300000),
(5, 3, 1, '2026-06-02', 1550000),
(1, 8, 4, '2026-06-10', 250000),
(2, 2, 2, '2026-07-06', 800000),
(1, 6, 2, '2026-09-03', 1100000),
(4, 7, 1, '2026-09-10', 1750000),
(5, 5, 4, '2026-09-13', 200000),
(1, 3, 2, '2026-09-21', 1200000);
```

Diseño del formulario

Control	Tipo	Uso
`cmbAgrupar`	ComboBox	Vendedor / Cliente / Categoría
`cmbMetrica`	ComboBox	Cantidad de ventas / Total vendido / Promedio por venta
`txtMinimo`	TextBox	El grupo debe superar este valor
`cmbAnio`	ComboBox	Todos / 2024 / 2025 / 2026
`btnFiltrar`	Button	Ejecutar
`dgvGrupos`	DataGridView	Grupo · Cantidad · Total · Promedio
`lblInfo`	Label	Mensajes

Requerimientos funcionales

1. El usuario elige cómo agrupar, qué métrica comparar y el valor mínimo.
2. Solo se muestran los grupos cuya métrica sea ESTRICTAMENTE mayor que el mínimo.
3. La métrica puede ser COUNT, SUM o AVG según cmbMetrica.
4. Si cmbAnio ≠ «Todos», se filtran primero las ventas de ese año (antes de agrupar) y luego los grupos.
5. La grilla siempre muestra Cantidad, Total y Promedio del grupo.
6. Si ningún grupo cumple, se informa «No hay grupos que cumplan la condición.»
7. El mínimo debe ser un número mayor o igual que cero.
8. La expresión de la columna y el agregado salen de listas cerradas, no del texto libre.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
GROUP BY + HAVING	Con COUNT(*), SUM(Total) y AVG(Total).
WHERE + GROUP BY + HAVING	Filtro de año previo a la agrupación.
INNER JOIN	Para los nombres.
ORDER BY	Total DESC.

Pistas (sin solución)

- HAVING puede usar el agregado directamente: no puedes referirte al alias definido en el mismo SELECT (en SQL Server).
- Si en un caso obtienes más filas de las esperadas, revisa si colocaste la condición en WHERE.
- El parámetro del mínimo debe ser decimal; COUNT devuelve int: comprueba que la comparación funcione para ambos.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Mínimo vacío	«Ingrese el valor mínimo.»
Mínimo = abc	«El valor mínimo debe ser un número válido.»
Mínimo negativo	«El valor mínimo no puede ser negativo.»
Ningún grupo cumple	«No hay grupos que cumplan la condición.»

Casos de prueba

Datos: los mismos 40 registros del Ejercicio 13. Vendedores: Laura 12 ventas, Mateo 9, Carolina 8, Jorge 6 y Diana 5.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Vendedores con más de 5 ventas	Vendedor · Cantidad de ventas · mín. 5 · Todos	4 filas (Diana Mora, con exactamente 5, NO aparece). Ver Tabla 14-A
CP-02	Vendedores con más de 8 ventas	Vendedor · Cantidad · mín. 8	2 filas: Laura Vega (12 ventas, $11.600.000), Mateo Ríos (9 ventas, $9.600.000)
CP-03	Categorías con total > $10.000.000	Categoría · Total vendido · mín. 10000000	2 filas. Ver Tabla 14-B
CP-04	Categorías con total > $20.000.000	Categoría · Total · mín. 20000000	1 fila(s): Tecnología (14 ventas, $20.150.000)
CP-05	Clientes con promedio > $1.000.000	Cliente · Promedio por venta · mín. 1000000	3 filas. Ver Tabla 14-C
CP-06	Combinar WHERE y HAVING	Vendedor · Cantidad · mín. 2 · Año 2026	3 filas. Ver Tabla 14-D
CP-07	Clientes con total > $5.000.000	Cliente · Total vendido · mín. 5000000 · Todos	2 filas: Papelería Central, Tienda Mayorista
CP-08	Ningún grupo cumple	Vendedor · Cantidad · mín. 100	0 filas; «No hay grupos que cumplan la condición.»
CP-09	Mínimo vacío	mín. (vacío)	«Ingrese el valor mínimo.»
CP-10	Mínimo negativo	mín. -5	«El valor mínimo no puede ser negativo.»

Tabla 14-A — Vendedores con más de 5 ventas

Grupo	Cantidad	Total	Promedio
Laura Vega	12	$11.600.000	$966.666,67
Mateo Ríos	9	$9.600.000	$1.066.666,67
Jorge Salas	6	$6.900.000	$1.150.000
Carolina Paz	8	$6.550.000	$818.750

Tabla 14-B — Categorías con total > 10.000.000

Categoría	Cantidad	Total	Promedio
Tecnología	14	$20.150.000	$1.439.285,71
Hogar	12	$12.600.000	$1.050.000

Tabla 14-C — Clientes con promedio > 1.000.000

Cliente	Cantidad	Total	Promedio
Papelería Central	9	$11.000.000	$1.222.222,22
Ofimática Cali	3	$4.550.000	$1.516.666,67
Distribuciones Sol	3	$3.300.000	$1.100.000

Tabla 14-D — Vendedores con más de 2 ventas en 2026

Grupo	Cantidad	Total	Promedio
Laura Vega	5	$4.900.000	$980.000
Diana Mora	4	$4.000.000	$1.000.000
Mateo Ríos	3	$3.550.000	$1.183.333,33

Reto adicional

Agrega una segunda condición HAVING (por ejemplo: cantidad > 3 Y total > $8.000.000) y compara el resultado con hacerlo en C# tras traer todos los grupos. Vendedores con cantidad > 3 y total > 8.000.000: Mateo Ríos, Laura Vega.

Espacio para notas

EJERCICIO 15 — CONSULTAS POR PERÍODOS DE TIEMPO

Nivel

Intermedio/Avanzado

Contexto

Una comercializadora quiere saber, en cualquier momento, cuánto ha vendido en distintos períodos: un año elegido, el año en curso, el mes actual, el mes pasado, los últimos 7 o 30 días, los últimos 10 o 12 meses, o un mes y año escogidos. Los períodos «actuales» deben calcularse solos a partir de la fecha del sistema.

Objetivo

Practicar las funciones de fecha de SQL Server (`GETDATE, DATEADD, DATEDIFF, YEAR, MONTH, DAY, DATEFROMPARTS, EOMONTH`), diferenciar un **período fijo** (2025) de un **período relativo** a la fecha actual (últimos 10 meses) y calcular COUNT, SUM, AVG, MIN y MAX sobre cada uno. Recuerda: SQL Server NO usa `NOW()`.

Base de datos

Nombre de la base de datos: `Ej15_Periodos`

Tabla `Ventas`

Columna	Tipo de datos	Restricciones
VentaId	INT	PK, autoincremental
FechaVenta	DATE	obligatorio
Cliente	NVARCHAR(60)	obligatorio
Vendedor	NVARCHAR(60)	obligatorio
Total	DECIMAL(14,2)	obligatorio

Relaciones: Una sola tabla. 54 ventas (6 de 2024, 18 de 2025, 27 de 2026 hasta el 30/09 y 3 posteriores: 05/10, 15/11 y 20/12 de 2026).

Script SQL

Ejecuta el script completo. Las fechas son fijas; los casos de prueba se calculan con fecha de referencia **30/09/2026**.

```
IF DB_ID(N'Ej15_Periodos') IS NOT NULL
BEGIN
    ALTER DATABASE Ej15_Periodos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej15_Periodos;
END
GO

CREATE DATABASE Ej15_Periodos;
GO

USE Ej15_Periodos;
GO

CREATE TABLE Ventas (
    VentaId     INT            IDENTITY(1,1) PRIMARY KEY,
    FechaVenta  DATE           NOT NULL,
    Cliente     NVARCHAR(60)   NOT NULL,
    Vendedor    NVARCHAR(60)   NOT NULL,
    Total       DECIMAL(14,2)  NOT NULL
);

INSERT INTO Ventas (FechaVenta, Cliente, Vendedor, Total) VALUES
    ('2024-03-15', N'Ferretería El Roble', N'Laura Vega', 600000),
    ('2024-06-20', N'Papelería Central', N'Mateo Ríos', 500000),
    ('2024-08-05', N'Comercial Andina', N'Mateo Ríos', 2750000),
    ('2024-10-12', N'Ferretería El Roble', N'Laura Vega', 2550000),
```

```
('2024-11-28', N'Servicios Delta', N'Carolina Paz', 2150000),
('2024-12-31', N'Ofimática Cali', N'Carolina Paz', 2600000),
('2025-01-10', N'Ferretería El Roble', N'Mateo Ríos', 2650000),
('2025-02-14', N'Tienda Mayorista', N'Mateo Ríos', 2350000),
('2025-03-08', N'Ofimática Cali', N'Mateo Ríos', 3300000),
('2025-03-22', N'Ofimática Cali', N'Jorge Salas', 900000),
('2025-04-17', N'Servicios Delta', N'Carolina Paz', 3200000),
('2025-05-09', N'Tienda Mayorista', N'Jorge Salas', 2950000),
('2025-05-30', N'Distribuciones Sol', N'Jorge Salas', 500000),
('2025-06-25', N'Ferretería El Roble', N'Mateo Ríos', 1500000),
('2025-07-04', N'Comercial Andina', N'Carolina Paz', 3550000),
('2025-08-19', N'Papelería Central', N'Laura Vega', 1100000),
('2025-09-30', N'Ofimática Cali', N'Laura Vega', 850000),
('2025-10-01', N'Servicios Delta', N'Carolina Paz', 2950000),
('2025-10-18', N'Ferretería El Roble', N'Mateo Ríos', 2650000),
('2025-11-05', N'Distribuciones Sol', N'Mateo Ríos', 2350000),
('2025-11-30', N'Supermercado La 14', N'Mateo Ríos', 2400000),
('2025-12-01', N'Ofimática Cali', N'Jorge Salas', 2550000),
('2025-12-15', N'Servicios Delta', N'Laura Vega', 2100000),
('2025-12-31', N'Servicios Delta', N'Mateo Ríos', 1100000),
('2026-01-02', N'Servicios Delta', N'Jorge Salas', 2800000),
('2026-01-15', N'Comercial Andina', N'Carolina Paz', 950000),
('2026-01-31', N'Supermercado La 14', N'Laura Vega', 650000),
('2026-02-10', N'Servicios Delta', N'Laura Vega', 2000000),
('2026-02-28', N'Papelería Central', N'Jorge Salas', 3450000),
('2026-03-05', N'Servicios Delta', N'Laura Vega', 750000),
('2026-03-19', N'Servicios Delta', N'Laura Vega', 3350000),
('2026-03-27', N'Ferretería El Roble', N'Jorge Salas', 2000000),
('2026-04-08', N'Distribuciones Sol', N'Laura Vega', 750000),
('2026-04-30', N'Tienda Mayorista', N'Jorge Salas', 3150000),
('2026-05-12', N'Tienda Mayorista', N'Laura Vega', 2300000),
('2026-05-21', N'Servicios Delta', N'Carolina Paz', 550000),
('2026-06-03', N'Ofimática Cali', N'Jorge Salas', 450000),
('2026-06-18', N'Tienda Mayorista', N'Jorge Salas', 400000),
('2026-07-07', N'Papelería Central', N'Carolina Paz', 1950000),
('2026-07-25', N'Servicios Delta', N'Carolina Paz', 2450000),
('2026-08-01', N'Ferretería El Roble', N'Carolina Paz', 3500000),
('2026-08-14', N'Tienda Mayorista', N'Laura Vega', 2650000),
('2026-08-30', N'Ferretería El Roble', N'Carolina Paz', 750000),
('2026-08-31', N'Papelería Central', N'Laura Vega', 2950000),
('2026-09-01', N'Tienda Mayorista', N'Mateo Ríos', 2500000),
('2026-09-10', N'Distribuciones Sol', N'Carolina Paz', 2550000),
('2026-09-22', N'Servicios Delta', N'Jorge Salas', 1150000),
('2026-09-23', N'Ofimática Cali', N'Laura Vega', 1000000),
('2026-09-25', N'Papelería Central', N'Jorge Salas', 1400000),
('2026-09-28', N'Ferretería El Roble', N'Carolina Paz', 2750000),
('2026-09-30', N'Ofimática Cali', N'Mateo Ríos', 1100000),
('2026-10-05', N'Servicios Delta', N'Mateo Ríos', 2550000),
('2026-11-15', N'Servicios Delta', N'Jorge Salas', 550000),
('2026-12-20', N'Supermercado La 14', N'Carolina Paz', 1200000);
```

Diseño del formulario

Control	Tipo	Uso
`cmbPeriodo`	ComboBox	Las 11 opciones de la tabla de períodos (A–K)
`txtAnio`	TextBox	Año (solo para opciones A y K)
`cmbMes`	ComboBox	Enero … Diciembre (solo opción K)
`dtpReferencia`	DateTimePicker	ShowCheckBox = true: si está marcado, reemplaza la fecha del sistema (modo prueba)
`btnConsultar`	Button	Consulta el período elegido
`lblPeriodo`	Label	«Del dd/MM/yyyy al dd/MM/yyyy»
`lblCantidad`	Label	COUNT
`lblTotal`	Label	SUM
`lblPromedio`	Label	AVG
`lblMinimo`	Label	MIN
`lblMaximo`	Label	MAX
`dgvVentas`	DataGridView	Ventas del período

Requerimientos funcionales

1. Un solo formulario resuelve las 11 opciones de cmbPeriodo; cada una calcula su propio rango de fechas.
2. Los períodos «actuales» (B a J) se calculan dinámicamente con la fecha del servidor (GETDATE()); no se escribe ninguna fecha fija en el código.
3. Si dtpReferencia está marcado, esa fecha sustituye a la del sistema (así puedes validar los casos en cualquier día real).
4. lblPeriodo muestra siempre la fecha inicial y final usadas por la consulta.
5. «Año actual hasta hoy» excluye las ventas posteriores a la fecha de referencia.
6. «Mes anterior» y «Año anterior» deben funcionar también cuando la referencia cae en enero.
7. «Últimos 10/12 meses» comienza el día 1 del mes correspondiente, sin usar fechas escritas a mano.
8. Se muestran cantidad, total, promedio (2 decimales), mínimo y máximo; sin ventas, todo vale 0.
9. La grilla lista las ventas del período, ordenadas por fecha.
10. Se validan año y mes cuando la opción los requiere.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
WHERE con BETWEEN o >= / <=	Filtro por rango fijo (A, K) y por rango calculado (B–J).
GETDATE(), DATEADD()	Períodos relativos: hoy, hace 7/30 días, hace 9/11 meses.
DATEFROMPARTS(), EOMONTH()	Primer y último día de año y mes.
YEAR(), MONTH()	Alternativa para años y meses (piensa cuál usa mejor los índices).
COUNT, SUM, AVG, MIN, MAX	Sobre cualquier período.
ISNULL	Sin ventas → 0.

Pistas (sin solución)

- Recomendación de diseño: primero calcula en SQL (o en C#) la FECHA INICIAL y la FECHA FINAL del período, y luego usa una única consulta de estadísticas para todos.
- `DATEADD(MONTH, -9, …)` aplicado a hoy NO da el día 1; combina DATEFROMPARTS con YEAR y MONTH para empezar en el día 1.
- Un período «móvil» (hoy menos 10 meses) y uno «calendario» (desde el día 1) dan resultados distintos: el reto te pide comprobarlo.
- Para «mes anterior en enero», piensa en restar un mes a una fecha, no en restar 1 al número del mes.
- Evita aplicar funciones a la columna en el WHERE (p. ej. YEAR(FechaVenta) = …) si quieres aprovechar índices; compara la columna con un rango.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Año vacío (opciones A y K)	«Ingrese el año.»
Año = abc	«El año debe ser un número válido.»
Año < 2000 o > 2100	«El año debe estar entre 2000 y 2100.»
Mes sin seleccionar (opción K)	«Seleccione un mes.»
Período sin ventas	«No hay ventas en el período seleccionado.»; todos los valores en 0
Sin opción elegida	«Seleccione un período.»

Casos de prueba

Referencia de todos los casos: **30/09/2026** (marca dtpReferencia con esa fecha o ejecuta el sistema en ese día). Las reglas de cada período son:

Definición de los períodos

Período	Regla	Rango con ref. 30/09/2026
Año seleccionado	Del 01/01 al 31/12 del año indicado	Año 2025: 01/01/2025 al 31/12/2025
Año actual	Año de la fecha de referencia, completo (incluye fechas futuras ya registradas)	01/01/2026 al 31/12/2026
Año actual hasta hoy	Del 01/01 del año actual hasta la fecha de referencia (inclusive)	01/01/2026 al 30/09/2026
Mes actual	Del primer al último día del mes de la fecha de referencia	01/09/2026 al 30/09/2026
Mes anterior	El mes calendario anterior (correcto también en enero)	01/08/2026 al 31/08/2026
Año anterior	El año calendario anterior, completo	01/01/2025 al 31/12/2025
Últimos 7 días	Desde (hoy − 7 días) hasta hoy, ambos inclusive	23/09/2026 al 30/09/2026
Últimos 30 días	Desde (hoy − 30 días) hasta hoy, ambos inclusive	31/08/2026 al 30/09/2026
Últimos 10 meses	Desde el día 1 del mes que está 9 meses antes del actual (10 meses calendario, incluido el actual) hasta hoy	01/12/2025 al 30/09/2026
Últimos 12 meses	Igual, con 12 meses calendario (11 meses antes del actual)	01/10/2025 al 30/09/2026
Mes y año elegidos	Del primer al último día del mes elegido	Agosto 2026: 01/08/2026 al 31/08/2026

ID	Acción	Datos de entrada	Resultado esperado
CP-01	A · Año seleccionado	Año: 2025	Cantidad 18 · Total $38.950.000 · Promedio $2.163.888,89 · Mínimo $500.000 · Máximo $3.550.000
CP-02	A · Año seleccionado	Año: 2026	Cantidad 30 · Total $54.550.000 · Promedio $1.818.333,33 · Mínimo $400.000 · Máximo $3.500.000
CP-03	A · Año seleccionado	Año: 2024	Cantidad 6 · Total $11.150.000 · Promedio $1.858.333,33 · Mínimo $500.000 · Máximo $2.750.000
CP-04	A · Año sin registros	Año: 2030	Cantidad 0 · Total $0 · Promedio $0 · Mínimo $0 · Máximo $0
CP-05	B · Año actual (ref. 30/09/2026)	Sin pedir año	Período 01/01/2026 al 31/12/2026. Cantidad 30 · Total $54.550.000 · Promedio $1.818.333,33 · Mínimo $400.000 · Máximo $3.500.000. Incluye las 3 ventas futuras (05/10, 15/11 y 20/12).
CP-06	C · Año actual hasta hoy	Ref. 30/09/2026	Período 01/01/2026 al 30/09/2026. Cantidad 27 · Total $50.250.000 · Promedio $1.861.111,11. Excluye las 3 ventas futuras.
CP-07	D · Mes actual	Ref. 30/09/2026	Período 01/09/2026 al 30/09/2026. Cantidad 7 · Total $12.450.000 · Promedio $1.778.571,43 · Mínimo $1.000.000 · Máximo $2.750.000
CP-08	E · Mes anterior	Ref. 30/09/2026	Período 01/08/2026 al 31/08/2026. Cantidad 4 · Total $9.850.000 · Promedio $2.462.500 · Mínimo $750.000 · Máximo $3.500.000
CP-09	E · Mes anterior en enero	Ref. simulada 15/01/2027	Período 01/12/2026 al 31/12/2026. Cantidad 1 · Total $1.200.000 · Promedio $1.200.000 · Mínimo $1.200.000 · Máximo $1.200.000
CP-10	E · Mes anterior en enero (otro año)	Ref. simulada 10/01/2026	Período 01/12/2025 al 31/12/2025. Cantidad 3 · Total $5.750.000 · Promedio $1.916.666,67 · Mínimo $1.100.000 · Máximo $2.550.000
CP-11	F · Año anterior	Ref. 30/09/2026	Período 01/01/2025 al 31/12/2025. Cantidad 18 · Total $38.950.000 · Promedio $2.163.888,89 · Mínimo $500.000 · Máximo $3.550.000
CP-12	F · Año anterior (otra referencia)	Ref. simulada 15/01/2027	Período 01/01/2026 al 31/12/2026 (año 2026). Cantidad 30 · Total $54.550.000 · Promedio $1.818.333,33 · Mínimo $400.000 · Máximo $3.500.000

ID	Acción	Datos de entrada	Resultado esperado
CP-13	G · Últimos 7 días	Ref. 30/09/2026	Período 23/09/2026 al 30/09/2026. Cantidad 4 · Total $6.250.000 · Promedio $1.562.500. Fechas: 23/09/2026, 25/09/2026, 28/09/2026, 30/09/2026 (el 22/09 queda fuera).
CP-14	H · Últimos 30 días	Ref. 30/09/2026	Período 31/08/2026 al 30/09/2026. Cantidad 8 · Total $15.400.000 · Promedio $1.925.000. El 30/08 queda fuera y el 31/08 entra.
CP-15	I · Últimos 10 meses	Ref. 30/09/2026	Fecha inicial 01/12/2025 · fecha final 30/09/2026. Cantidad 30 · Total $56.000.000 · Promedio $1.866.666,67
CP-16	I · Últimos 10 meses	Ref. simulada 15/03/2026	Fecha inicial 01/06/2025 · fecha final 15/03/2026. Cantidad 17 · Total $33.700.000 · Promedio $1.982.352,94
CP-17	I · Borde del período	Ref. 30/09/2026	La venta del 01/12/2025 SÍ entra y la del 30/11/2025 NO entra en los últimos 10 meses.
CP-18	J · Últimos 12 meses	Ref. 30/09/2026	Fecha inicial 01/10/2025 · fecha final 30/09/2026. Cantidad 34 · Total $66.350.000 · Promedio $1.951.470,59
CP-19	K · Mes y año seleccionados	Año 2026 · Mes Agosto	Cantidad 4 · Total $9.850.000 · Promedio $2.462.500 · Mínimo $750.000 · Máximo $3.500.000
CP-20	K · Mes y año seleccionados	Año 2026 · Mes Marzo	Cantidad 3 · Total $6.100.000 · Promedio $2.033.333,33 · Mínimo $750.000 · Máximo $3.350.000
CP-21	K · Mes y año seleccionados	Año 2026 · Mes Febrero	Cantidad 2 · Total $5.450.000 · Promedio $2.725.000 · Mínimo $2.000.000 · Máximo $3.450.000
CP-22	K · Mes y año seleccionados	Año 2026 · Mes Octubre	Cantidad 1 · Total $2.550.000 · Promedio $2.550.000 · Mínimo $2.550.000 · Máximo $2.550.000
CP-23	K · Mes y año seleccionados	Año 2025 · Mes Diciembre	Cantidad 3 · Total $5.750.000 · Promedio $1.916.666,67 · Mínimo $1.100.000 · Máximo $2.550.000
CP-24	K · Mes sin registros	Año 2030 · Mes Enero	Cantidad 0 · Total $0 · Promedio $0 · Mínimo $0 · Máximo $0
CP-25	K · Mes no seleccionado	Año 2026 · Mes (vacío)	«Seleccione un mes.»
CP-26	K · Año inválido	Año abc · Mes Marzo	«El año debe ser un número válido.»

Reto adicional

Implementa «Últimos 10 meses (móvil)»: desde `DATEADD(MONTH, -10, hoy)` hasta hoy. Con referencia 30/09/2026 el rango es 30/11/2025 al 30/09/2026 y debes obtener 31 ventas (una más que con el período calendario; total $58.400.000). Explica en tus notas por qué la diferencia.

Espacio para notas

EJERCICIO 16 — REPORTES Y ESTADÍSTICAS POR PERÍODO

Nivel

Intermedio/Avanzado

Contexto

La gerencia necesita reportes tabulares: ventas mes a mes de un año, los últimos 10 meses en detalle, una comparación entre el año anterior y el actual, y el desempeño de cada vendedor en un año.

Objetivo

Combinar **GROUP BY** con funciones de fecha (YEAR, MONTH, DATEADD, DATEFROMPARTS), COUNT, SUM y AVG para producir reportes en un DataGridView, y resolver una comparación entre dos períodos en una sola consulta.

Base de datos

Nombre de la base de datos: `Ej16_ReportesPeriodo`

Tabla `Ventas`

Columna	Tipo de datos	Restricciones
VentaId	INT	PK, autoincremental
FechaVenta	DATE	obligatorio
Cliente	NVARCHAR(60)	obligatorio
Vendedor	NVARCHAR(60)	obligatorio
Total	DECIMAL(14,2)	obligatorio

Relaciones: Una sola tabla (la misma del Ejercicio 15).

Script SQL

Mismos datos del Ejercicio 15. Fecha de referencia de los casos: **30/09/2026**.

```
IF DB_ID(N'Ej16_ReportesPeriodo') IS NOT NULL
BEGIN
    ALTER DATABASE Ej16_ReportesPeriodo SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej16_ReportesPeriodo;
END
GO

CREATE DATABASE Ej16_ReportesPeriodo;
GO

USE Ej16_ReportesPeriodo;
GO

CREATE TABLE Ventas (
    VentaId     INT            IDENTITY(1,1) PRIMARY KEY,
    FechaVenta  DATE           NOT NULL,
    Cliente     NVARCHAR(60)   NOT NULL,
    Vendedor    NVARCHAR(60)   NOT NULL,
    Total       DECIMAL(14,2)  NOT NULL
);

INSERT INTO Ventas (FechaVenta, Cliente, Vendedor, Total) VALUES
    ('2024-03-15', N'Ferretería El Roble', N'Laura Vega', 600000),
    ('2024-06-20', N'Papelería Central', N'Mateo Ríos', 500000),
    ('2024-08-05', N'Comercial Andina', N'Mateo Ríos', 2750000),
    ('2024-10-12', N'Ferretería El Roble', N'Laura Vega', 2550000),
    ('2024-11-28', N'Servicios Delta', N'Carolina Paz', 2150000),
    ('2024-12-31', N'Ofimática Cali', N'Carolina Paz', 2600000),
    ('2025-01-10', N'Ferretería El Roble', N'Mateo Ríos', 2650000),
    ('2025-02-14', N'Tienda Mayorista', N'Mateo Ríos', 2350000),
    ('2025-03-08', N'Ofimática Cali', N'Mateo Ríos', 3300000),
    ('2025-03-22', N'Ofimática Cali', N'Jorge Salas', 900000),
```

```
('2025-04-17', N'Servicios Delta', N'Carolina Paz', 3200000),
('2025-05-09', N'Tienda Mayorista', N'Jorge Salas', 2950000),
('2025-05-30', N'Distribuciones Sol', N'Jorge Salas', 500000),
('2025-06-25', N'Ferretería El Roble', N'Mateo Ríos', 1500000),
('2025-07-04', N'Comercial Andina', N'Carolina Paz', 3550000),
('2025-08-19', N'Papelería Central', N'Laura Vega', 1100000),
('2025-09-30', N'Ofimática Cali', N'Laura Vega', 850000),
('2025-10-01', N'Servicios Delta', N'Carolina Paz', 2950000),
('2025-10-18', N'Ferretería El Roble', N'Mateo Ríos', 2650000),
('2025-11-05', N'Distribuciones Sol', N'Mateo Ríos', 2350000),
('2025-11-30', N'Supermercado La 14', N'Mateo Ríos', 2400000),
('2025-12-01', N'Ofimática Cali', N'Jorge Salas', 2550000),
('2025-12-15', N'Servicios Delta', N'Laura Vega', 2100000),
('2025-12-31', N'Servicios Delta', N'Mateo Ríos', 1100000),
('2026-01-02', N'Servicios Delta', N'Jorge Salas', 2800000),
('2026-01-15', N'Comercial Andina', N'Carolina Paz', 950000),
('2026-01-31', N'Supermercado La 14', N'Laura Vega', 650000),
('2026-02-10', N'Servicios Delta', N'Laura Vega', 2000000),
('2026-02-28', N'Papelería Central', N'Jorge Salas', 3450000),
('2026-03-05', N'Servicios Delta', N'Laura Vega', 750000),
('2026-03-19', N'Servicios Delta', N'Laura Vega', 3350000),
('2026-03-27', N'Ferretería El Roble', N'Jorge Salas', 2000000),
('2026-04-08', N'Distribuciones Sol', N'Laura Vega', 750000),
('2026-04-30', N'Tienda Mayorista', N'Jorge Salas', 3150000),
('2026-05-12', N'Tienda Mayorista', N'Laura Vega', 2300000),
('2026-05-21', N'Servicios Delta', N'Carolina Paz', 550000),
('2026-06-03', N'Ofimática Cali', N'Jorge Salas', 450000),
('2026-06-18', N'Tienda Mayorista', N'Jorge Salas', 400000),
('2026-07-07', N'Papelería Central', N'Carolina Paz', 1950000),
('2026-07-25', N'Servicios Delta', N'Carolina Paz', 2450000),
('2026-08-01', N'Ferretería El Roble', N'Carolina Paz', 3500000),
('2026-08-14', N'Tienda Mayorista', N'Laura Vega', 2650000),
('2026-08-30', N'Ferretería El Roble', N'Carolina Paz', 750000),
('2026-08-31', N'Papelería Central', N'Laura Vega', 2950000),
('2026-09-01', N'Tienda Mayorista', N'Mateo Ríos', 2500000),
('2026-09-10', N'Distribuciones Sol', N'Carolina Paz', 2550000),
('2026-09-22', N'Servicios Delta', N'Jorge Salas', 1150000),
('2026-09-23', N'Ofimática Cali', N'Laura Vega', 1000000),
('2026-09-25', N'Papelería Central', N'Jorge Salas', 1400000),
('2026-09-28', N'Ferretería El Roble', N'Carolina Paz', 2750000),
('2026-09-30', N'Ofimática Cali', N'Mateo Ríos', 1100000),
('2026-10-05', N'Servicios Delta', N'Mateo Ríos', 2550000),
('2026-11-15', N'Servicios Delta', N'Jorge Salas', 550000),
('2026-12-20', N'Supermercado La 14', N'Carolina Paz', 1200000);
```

Diseño del formulario

Control	Tipo	Uso
`cmbReporte`	ComboBox	Ventas por mes del año / Últimos 10 meses por mes / Año anterior vs año actual / Ventas por vendedor en un año
`txtAnio`	TextBox	Año (reportes 1 y 4)
`dtpReferencia`	DateTimePicker	ShowCheckBox = true; modo prueba
`btnGenerar`	Button	Genera el reporte
`dgvReporte`	DataGridView	Resultado del reporte
`lblTotal`	Label	Suma de la columna Total

Requerimientos funcionales

1. El ComboBox selecciona uno de los cuatro reportes.
2. «Ventas por mes del año» muestra un renglón por cada mes CON ventas, con el nombre del mes en español, en orden de calendario.
3. «Últimos 10 meses» muestra 10 renglones (del mes 9 meses atrás al mes actual), en orden cronológico, identificados con año y mes.
4. «Año anterior vs año actual» muestra dos renglones con Cantidad, Total y Promedio.
5. «Ventas por vendedor» muestra Cantidad, Total y Promedio por vendedor del año elegido, de mayor a menor Total.

6 Todos los promedios se muestran redondeados a 2 decimales y los valores monetarios con separador de miles.
7 lblTotal suma la columna Total del reporte mostrado.
8 Si no hay datos, la grilla queda vacía y se informa al usuario.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
GROUP BY YEAR, MONTH	Reporte mensual de un año.
COUNT, SUM, AVG	En los cuatro reportes.
GETDATE(), DATEADD(), DATEFROMPARTS()	Rango de los últimos 10 meses.
Comparación en una consulta	Piensa en SUM/COUNT condicionales (CASE) o en UNION ALL de dos consultas.
ORDER BY	Por mes / total.

Pistas (sin solución)

- Filtra el rango de fechas en el WHERE y agrupa después por YEAR y MONTH (los dos, para que el reporte de 10 meses no mezcle años).
- Si ordenas por el nombre del mes verás «Abril» antes que «Enero»: ordena por el NÚMERO del mes.
- Traducir el número de mes a texto puede hacerse en SQL (CASE) o al llenar la grilla en C#.
- Reto de pensamiento: ¿qué pasa con los meses sin ventas? GROUP BY no los muestra.

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Año vacío / abc / fuera de 2000–2100	Mensajes del Ejercicio 15 («Ingrese el año.», «El año debe ser un número válido.», «El año debe estar entre 2000 y 2100.»)
Reporte no elegido	«Seleccione un reporte.»
Año sin ventas	«No hay ventas registradas en el año 2030.»

Casos de prueba

Referencia: 30/09/2026. Los promedios están redondeados a 2 decimales.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	L · Ventas por mes del año	Año: 2026	12 filas. Ver Tabla 16-A
CP-02	L · Ventas por mes del año	Año: 2025	12 filas. Ver Tabla 16-B
CP-03	L · Ventas por mes del año	Año: 2024	6 filas (solo meses con ventas). Ver Tabla 16-C
CP-04	L · Año sin ventas	Año: 2030	0 filas; «No hay ventas registradas en el año 2030.»
CP-05	M · Últimos 10 meses agrupados	Ref. 30/09/2026	10 filas (2025-12 a 2026-09); suma de la columna Total = $56.000.000. Ver Tabla 16-D
CP-06	N · Comparación de períodos	Año anterior vs año actual (completos)	2 filas. Ver Tabla 16-E
CP-07	N · Comparación hasta la fecha	Ene–Sep 2025 vs Ene–Sep 2026 (reto)	2025: Cantidad 11 · Total $22.850.000 · Promedio $2.077.272,73 · 2026: Cantidad 27 · Total $50.250.000 · Promedio $1.861.111,11
CP-08	O · Ventas por vendedor	Año: 2026	4 filas. Ver Tabla 16-F
CP-09	O · Ventas por vendedor	Año: 2025	4 filas. Ver Tabla 16-G
CP-10	O · Año vacío	Año (vacío)	«Ingrese el año.»

ID	Acción	Datos de entrada	Resultado esperado
CP-11	M · Orden cronológico	Ref. 30/09/2026	La primera fila es diciembre de 2025 y la última septiembre de 2026 (no alfabético).

Tabla 16-A — Ventas por mes, 2026

Mes	Cantidad	Total	Promedio
01 · Enero	3	$4.400.000	$1.466.666,67
02 · Febrero	2	$5.450.000	$2.725.000
03 · Marzo	3	$6.100.000	$2.033.333,33
04 · Abril	2	$3.900.000	$1.950.000
05 · Mayo	2	$2.850.000	$1.425.000
06 · Junio	2	$850.000	$425.000
07 · Julio	2	$4.400.000	$2.200.000
08 · Agosto	4	$9.850.000	$2.462.500
09 · Septiembre	7	$12.450.000	$1.778.571,43
10 · Octubre	1	$2.550.000	$2.550.000
11 · Noviembre	1	$550.000	$550.000
12 · Diciembre	1	$1.200.000	$1.200.000

Tabla 16-B — Ventas por mes, 2025

Mes	Cantidad	Total	Promedio
01 · Enero	1	$2.650.000	$2.650.000
02 · Febrero	1	$2.350.000	$2.350.000
03 · Marzo	2	$4.200.000	$2.100.000
04 · Abril	1	$3.200.000	$3.200.000
05 · Mayo	2	$3.450.000	$1.725.000
06 · Junio	1	$1.500.000	$1.500.000
07 · Julio	1	$3.550.000	$3.550.000
08 · Agosto	1	$1.100.000	$1.100.000
09 · Septiembre	1	$850.000	$850.000
10 · Octubre	2	$5.600.000	$2.800.000
11 · Noviembre	2	$4.750.000	$2.375.000
12 · Diciembre	3	$5.750.000	$1.916.666,67

Tabla 16-C — Ventas por mes, 2024

Mes	Cantidad	Total	Promedio
03 · Marzo	1	$600.000	$600.000
06 · Junio	1	$500.000	$500.000
08 · Agosto	1	$2.750.000	$2.750.000
10 · Octubre	1	$2.550.000	$2.550.000
11 · Noviembre	1	$2.150.000	$2.150.000
12 · Diciembre	1	$2.600.000	$2.600.000

Tabla 16-D — Últimos 10 meses (ref. 30/09/2026)

Mes	Cantidad	Total	Promedio
2025-12 Diciembre	3	$5.750.000	$1.916.666,67
2026-01 Enero	3	$4.400.000	$1.466.666,67
2026-02 Febrero	2	$5.450.000	$2.725.000
2026-03 Marzo	3	$6.100.000	$2.033.333,33
2026-04 Abril	2	$3.900.000	$1.950.000
2026-05 Mayo	2	$2.850.000	$1.425.000
2026-06 Junio	2	$850.000	$425.000
2026-07 Julio	2	$4.400.000	$2.200.000
2026-08 Agosto	4	$9.850.000	$2.462.500
2026-09 Septiembre	7	$12.450.000	$1.778.571,43

Tabla 16-E — Comparación de períodos

Período	Cantidad	Total	Promedio
Año anterior (2025)	18	$38.950.000	$2.163.888,89
Año actual (2026)	30	$54.550.000	$1.818.333,33

Tabla 16-F — Por vendedor, 2026

Vendedor	Cantidad	Total	Promedio
Carolina Paz	9	$16.650.000	$1.850.000
Laura Vega	9	$16.400.000	$1.822.222,22
Jorge Salas	9	$15.350.000	$1.705.555,56
Mateo Ríos	3	$6.150.000	$2.050.000

Tabla 16-G — Por vendedor, 2025

Vendedor	Cantidad	Total	Promedio
Mateo Ríos	8	$18.300.000	$2.287.500
Carolina Paz	3	$9.700.000	$3.233.333,33
Jorge Salas	4	$6.900.000	$1.725.000
Laura Vega	3	$4.050.000	$1.350.000

Reto adicional

Muestra TAMBIÉN los meses sin ventas (con cantidad 0 y total $0) en el reporte de un año: por ejemplo, 2024 debe mostrar 12 filas, 6 de ellas en cero. Investiga una tabla de números/calendario o un CTE recursivo. Añade un último renglón «Total» y comprueba que 2025 suma $38.950.000 y 2026 suma $54.550.000.

Espacio para notas

7. Nivel 4 — Ejercicios avanzados y proyecto final

Un sistema con cuatro tablas, filtros combinados, reportes y manejo de errores; y el proyecto integrador de toda la guía.

N.º	Ejercicio	Nivel	Casos de prueba
17	SISTEMA CON MÚLTIPLES FILTROS Y REPORTES	Avanzado	13
18	PROYECTO FINAL: SISTEMA DE GESTIÓN COMERCIAL	Avanzado	15

EJERCICIO 17 — SISTEMA CON MÚLTIPLES FILTROS Y REPORTES

Nivel

Avanzado

Contexto

Una distribuidora de tecnología registra cada venta con varios productos. Gerencia quiere una sola pantalla para consultar el detalle de ventas con cualquier combinación de filtros y además obtener tres reportes: total por cliente, productos más vendidos y categorías que superan cierto valor.

Objetivo

Unir **cuatro tablas** (Clientes, Ventas, DetalleVentas, Productos) con INNER JOIN, combinar filtros opcionales, calcular subtotales (Cantidad × PrecioUnitario) y producir reportes con GROUP BY, HAVING, TOP, SUM y COUNT. Este ejercicio exige manejo de errores con SqlException.

Base de datos

Nombre de la base de datos: `Ej17_SistemaVentas`

Tabla `Clientes`

Columna	Tipo de datos	Restricciones
ClienteId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
Ciudad	NVARCHAR(50)	obligatorio

Tabla `Productos`

Columna	Tipo de datos	Restricciones
ProductoId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
Categoria	NVARCHAR(40)	obligatorio
Precio	DECIMAL(12,2)	obligatorio

Tabla `Ventas`

Columna	Tipo de datos	Restricciones
VentaId	INT	PK, autoincremental
ClienteId	INT	FK → Clientes(ClienteId), obligatorio
FechaVenta	DATE	obligatorio

Tabla `DetalleVentas`

Columna	Tipo de datos	Restricciones
DetalleId	INT	PK, autoincremental
VentaId	INT	FK → Ventas(VentaId), obligatorio
ProductoId	INT	FK → Productos(ProductoId), obligatorio
Cantidad	INT	obligatorio, valida Cantidad > 0
PrecioUnitario	DECIMAL(12,2)	obligatorio

Relaciones: Ventas.ClienteId → Clientes; DetalleVentas.VentaId → Ventas; DetalleVentas.ProductoId → Productos. Una venta tiene 1 a 3 líneas de detalle.

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej17_SistemaVentas') IS NOT NULL
BEGIN
    ALTER DATABASE Ej17_SistemaVentas SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej17_SistemaVentas;
END
GO

CREATE DATABASE Ej17_SistemaVentas;
GO

USE Ej17_SistemaVentas;
GO

CREATE TABLE Clientes (
    ClienteId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre     NVARCHAR(80)  NOT NULL,
    Ciudad     NVARCHAR(50)  NOT NULL
);

CREATE TABLE Productos (
    ProductoId  INT            IDENTITY(1,1) PRIMARY KEY,
    Nombre      NVARCHAR(80)   NOT NULL,
    Categoria   NVARCHAR(40)   NOT NULL,
    Precio      DECIMAL(12,2)  NOT NULL
);

CREATE TABLE Ventas (
    VentaId     INT   IDENTITY(1,1) PRIMARY KEY,
    ClienteId   INT   NOT NULL REFERENCES Clientes(ClienteId),
    FechaVenta  DATE  NOT NULL
);

CREATE TABLE DetalleVentas (
    DetalleId       INT            IDENTITY(1,1) PRIMARY KEY,
    VentaId         INT            NOT NULL REFERENCES Ventas(VentaId),
    ProductoId      INT            NOT NULL REFERENCES Productos(ProductoId),
    Cantidad        INT            NOT NULL CHECK (Cantidad > 0),
    PrecioUnitario  DECIMAL(12,2)  NOT NULL
);

INSERT INTO Clientes (Nombre, Ciudad) VALUES
    (N'Andrea Molina', N'Cali'),
    (N'Bernardo Cruz', N'Bogotá'),
    (N'Carla Ramos', N'Medellín'),
    (N'Daniel Ortega', N'Cali'),
    (N'Elena Sandoval', N'Bogotá'),
    (N'Fernando Bravo', N'Barranquilla'),
    (N'Gloria Patiño', N'Medellín'),
    (N'Hugo Vanegas', N'Pereira');

INSERT INTO Productos (Nombre, Categoria, Precio) VALUES
    (N'Mouse Óptico', N'Periféricos', 35000),
    (N'Teclado Mecánico', N'Periféricos', 185000),
    (N'Webcam HD', N'Periféricos', 120000),
    (N'Monitor 24 pulgadas', N'Pantallas', 690000),
    (N'Monitor 32 pulgadas', N'Pantallas', 1290000),
    (N'Disco Duro 1TB', N'Almacenamiento', 260000),
    (N'Memoria USB 32GB', N'Almacenamiento', 28000),
    (N'Router WiFi', N'Redes', 145000),
    (N'Switch 8 Puertos', N'Redes', 98000),
    (N'Impresora Láser', N'Impresión', 780000),
    (N'Tóner Negro', N'Impresión', 210000),
    (N'Portátil 15 pulgadas', N'Computadores', 2890000);

INSERT INTO Ventas (ClienteId, FechaVenta) VALUES
    (4, '2025-06-24'),
    (8, '2025-08-20'),
    (2, '2025-09-17'),
    (5, '2025-10-06'),
    (4, '2025-10-17'),
    (6, '2025-11-11'),
    (2, '2025-11-17'),
```

```
    (7, '2025-11-20'),
    (3, '2025-12-03'),
    (7, '2025-12-30'),
    (5, '2026-01-02'),
    (3, '2026-02-18'),
    (5, '2026-02-25'),
    (7, '2026-04-15'),
    (1, '2026-04-17'),
    (6, '2026-04-22'),
    (6, '2026-04-22'),
    (5, '2026-05-11'),
    (6, '2026-05-18'),
    (2, '2026-05-21'),
    (1, '2026-05-30'),
    (1, '2026-06-01'),
    (1, '2026-06-05'),
    (8, '2026-06-08'),
    (8, '2026-06-16'),
    (3, '2026-06-19'),
    (6, '2026-07-02'),
    (1, '2026-07-29'),
    (7, '2026-08-03'),
    (7, '2026-08-09');

INSERT INTO DetalleVentas (VentaId, ProductoId, Cantidad, PrecioUnitario) VALUES
    (1, 4, 3, 690000),
    (1, 8, 5, 145000),
    (1, 9, 1, 98000),
    (2, 12, 2, 2890000),
    (2, 1, 1, 35000),
    (3, 10, 1, 780000),
    (4, 10, 1, 780000),
    (4, 4, 5, 690000),
    (5, 5, 2, 1290000),
    (6, 1, 4, 35000),
    (7, 1, 3, 35000),
    (7, 12, 2, 2890000),
    (7, 6, 2, 260000),
    (8, 1, 1, 35000),
    (8, 2, 1, 185000),
    (8, 11, 1, 210000),
    (9, 1, 2, 35000),
    (9, 6, 2, 260000),
    (9, 5, 2, 1290000),
    (10, 12, 5, 2890000),
    (10, 1, 1, 35000),
    (10, 7, 2, 28000),
    (11, 1, 1, 35000),
    (12, 10, 1, 780000),
    (12, 11, 3, 210000),
    (13, 8, 3, 145000),
    (13, 1, 4, 35000),
    (14, 10, 4, 780000),
    (14, 1, 5, 35000),
    (14, 5, 2, 1290000),
    (15, 4, 3, 690000),
    (15, 2, 1, 185000),
    (16, 8, 2, 145000),
    (17, 10, 5, 780000),
    (17, 7, 3, 28000),
    (17, 8, 2, 145000),
    (18, 5, 5, 1290000),
    (18, 12, 4, 2890000),
    (19, 1, 1, 35000),
    (19, 9, 3, 98000),
    (19, 3, 1, 120000),
    (20, 3, 2, 120000),
    (21, 8, 2, 145000),
    (22, 12, 2, 2890000),
    (22, 1, 4, 35000),
    (22, 4, 1, 690000),
    (23, 2, 2, 185000),
    (23, 10, 5, 780000),
    (24, 12, 4, 2890000),
    (24, 6, 3, 260000),
    (24, 5, 5, 1290000),
    (25, 3, 1, 120000),
```

```
(26, 7, 1, 28000),
(26, 3, 5, 120000),
(27, 2, 1, 185000),
(27, 4, 1, 690000),
(27, 12, 2, 2890000),
(28, 2, 2, 185000),
(29, 9, 4, 98000),
(30, 5, 4, 1290000),
(30, 9, 2, 98000);
```

Diseño del formulario

Control	Tipo	Uso
`cmbCliente`	ComboBox	Todos + 8 clientes
`cmbCategoria`	ComboBox	Todas + 6 categorías
`txtProducto`	TextBox	Nombre del producto (contiene)
`dtpDesde`	DateTimePicker	ShowCheckBox = true
`dtpHasta`	DateTimePicker	ShowCheckBox = true
`btnConsultar`	Button	Detalle de ventas
`btnRepClientes`	Button	Total por cliente
`btnRepProductos`	Button	Top 5 productos
`btnRepCategorias`	Button	Categorías sobre el mínimo
`txtMinimoCategoria`	TextBox	Valor mínimo (reporte de categorías)
`dgvDatos`	DataGridView	Detalle o reporte
`lblResumen`	Label	Líneas, ventas, total y unidades

Requerimientos funcionales

1. El detalle muestra: Cliente, Fecha, Producto, Categoría, Cantidad, Precio unitario y Subtotal.
2. Todos los filtros son opcionales y se combinan con AND.
3. El producto se busca por coincidencia parcial; cliente y categoría por igualdad.
4. Las fechas filtran solo cuando su casilla está marcada (extremos incluidos).
5. lblResumen muestra número de líneas, número de ventas distintas, total y unidades.
6. El reporte por cliente muestra Cliente, Ventas (cantidad de ventas distintas) y Total, de mayor a menor.
7. El reporte Top 5 muestra los 5 productos con más unidades vendidas.
8. El reporte de categorías usa el valor de txtMinimoCategoria (por defecto 10.000.000) y muestra solo categorías cuyo total lo supere.
9. Los reportes respetan los filtros de cliente y fechas cuando estén diligenciados.
10. Los errores de conexión, de SQL y de conversión se informan sin cerrar la aplicación.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
INNER JOIN múltiple	Ventas ↔ Clientes ↔ DetalleVentas ↔ Productos.
WHERE dinámico	Cinco filtros opcionales.
SUM(Cantidad * PrecioUnitario)	Subtotales y totales.
COUNT(DISTINCT)	Ventas distintas.
GROUP BY + HAVING	Reporte de categorías.
TOP 5 + ORDER BY	Productos más vendidos.

Pistas (sin solución)

- Dibuja las cuatro tablas y las llaves antes de escribir el JOIN; un JOIN mal encadenado multiplica filas.
- Si COUNT(*) te da el número de líneas, ¿cómo cuentas las ventas? Investiga COUNT(DISTINCT …).
- El precio de la venta es el de DetalleVentas.PrecioUnitario, no el de Productos.Precio (aunque hoy coincidan).

- Construye cada reporte primero en SSMS y compara con la tabla esperada; después pásalo a C#.
- Captura `SqlException` (errores de servidor/SQL), `FormatException` (conversión) e `InvalidOperationException` (conexión ya abierta).

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Fecha inicial > final	«La fecha inicial no puede ser mayor que la fecha final.»
Mínimo de categoría = abc	«El valor mínimo debe ser un número válido.»
Mínimo negativo	«El valor mínimo no puede ser negativo.»
Sin resultados	«No se encontraron ventas con los filtros indicados.»
Servidor no disponible	«No se pudo conectar con la base de datos. Verifique el servidor.»
Error SQL inesperado	«Error de base de datos: » + mensaje (sin cerrar la aplicación)

Casos de prueba

Datos: 8 clientes, 12 productos, 30 ventas y 61 líneas de detalle (enero 2025 a agosto 2026).

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Consulta sin filtros	Todos los filtros vacíos	61 líneas de detalle en 30 ventas; total $117.878.000; unidades 155
CP-02	Filtrar por cliente	Cliente: Andrea Molina	9 líneas de detalle en 5 ventas; total $13.795.000; unidades 22
CP-03	Filtrar por categoría	Categoría: Pantallas	11 líneas de detalle en 11 ventas; total $34.770.000; unidades 33
CP-04	Buscar producto (LIKE)	Producto: usb	3 líneas de detalle en 3 ventas; total $168.000; unidades 6
CP-05	Rango de fechas	Desde 01/01/2026 · Hasta 31/12/2026	39 líneas de detalle en 20 ventas; total $76.894.000; unidades 106
CP-06	Cliente + categoría	Cliente: Andrea Molina · Categoría: Periféricos	4 líneas de detalle en 4 ventas; total $1.065.000; unidades 9
CP-07	Categoría + fechas + producto	Categoría: Periféricos · Producto: mouse · Desde 01/01/2025 · Hasta 31/12/2025	6 líneas de detalle en 6 ventas; total $420.000; unidades 12
CP-08	Combinación sin resultados	Cliente: Andrea Molina · Categoría: Almacenamiento	0 líneas; «No se encontraron ventas con los filtros indicados.»
CP-09	Reporte: total por cliente	Sin filtros	Ver Tabla 17-A
CP-10	Reporte: Top 5 productos	Sin filtros	Ver Tabla 17-B (por unidades vendidas)
CP-11	Reporte: categorías con total > $10.000.000	Sin filtros	Ver Tabla 17-C
CP-12	Fechas invertidas	Desde 01/06/2026 · Hasta 01/01/2026	«La fecha inicial no puede ser mayor que la fecha final.»
CP-13	Verificación del subtotal	Sin filtros	La suma de Cantidad × PrecioUnitario de todas las líneas es $117.878.000

Tabla 17-A — Total por cliente (orden Total DESC)

Cliente	Ventas	Total
Gloria Patiño	5	$26.594.000
Hugo Vanegas	3	$24.725.000
Elena Sandoval	4	$22.850.000
Andrea Molina	5	$13.795.000
Fernando Bravo	5	$11.808.000
Bernardo Cruz	3	$7.425.000
Daniel Ortega	2	$5.473.000
Carla Ramos	3	$5.208.000

Tabla 17-B — Top 5 productos por unidades

Producto	Unidades
Mouse Óptico	27
Portátil 15 pulgadas	21
Monitor 32 pulgadas	20
Impresora Láser	17
Router WiFi	14

Tabla 17-C — Categorías con total > $10.000.000

Categoría	Total
Computadores	$60.690.000
Pantallas	$34.770.000
Impresión	$14.100.000

Reto adicional

Agrega un ComboBox «Período» (Año actual, Mes actual, Últimos 30 días, Últimos 10 meses) que rellene las fechas usando GETDATE(). Con la fecha 30/09/2026, «Últimos 10 meses» (01/12/2025 al 30/09/2026) debe dar: 45 líneas de detalle en 22 ventas; total $94.605.000; unidades 120.

Espacio para notas

EJERCICIO 18 — PROYECTO FINAL: SISTEMA DE GESTIÓN COMERCIAL

Nivel

Avanzado

Contexto

Una tienda de tecnología, hogar, oficina y deportes necesita un sistema de escritorio completo: registrar, consultar, corregir y eliminar ventas; buscarlas por nombre de cliente; filtrarlas por múltiples criterios y fechas; y obtener estadísticas por períodos y por grupos.

Objetivo

Integrar TODO lo aprendido: CRUD, LIKE, filtros múltiples, DateTimePicker, ComboBox, INNER JOIN de 4 tablas, GROUP BY, HAVING, TOP, SUM, AVG, COUNT, MIN, MAX, consultas por año, mes, año y mes actuales, año y mes anteriores, últimos 30 días y últimos 10 meses, con validaciones y manejo de errores.

Base de datos

Nombre de la base de datos: `Ej18_ProyectoFinal`

Tabla `Categorias`

Columna	Tipo de datos	Restricciones
CategoriaId	INT	PK, autoincremental
Nombre	NVARCHAR(40)	obligatorio, único

Tabla `Productos`

Columna	Tipo de datos	Restricciones
ProductoId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
CategoriaId	INT	FK → Categorias(CategoriaId), obligatorio
Precio	DECIMAL(12,2)	obligatorio, valida Precio > 0
Stock	INT	obligatorio, valida Stock >= 0

Tabla `Clientes`

Columna	Tipo de datos	Restricciones
ClienteId	INT	PK, autoincremental
Nombre	NVARCHAR(80)	obligatorio
Ciudad	NVARCHAR(50)	obligatorio

Tabla `Vendedores`

Columna	Tipo de datos	Restricciones
VendedorId	INT	PK, autoincremental
Nombre	NVARCHAR(60)	obligatorio
Zona	NVARCHAR(20)	obligatorio

Tabla `Ventas`

Columna	Tipo de datos	Restricciones
VentaId	INT	PK, autoincremental
ClienteId	INT	FK → Clientes(ClienteId), obligatorio
ProductoId	INT	FK → Productos(ProductoId), obligatorio
VendedorId	INT	FK → Vendedores(VendedorId), obligatorio
Cantidad	INT	obligatorio, valida Cantidad > 0
FechaVenta	DATE	obligatorio
Total	DECIMAL(14,2)	obligatorio

Relaciones: Productos.CategoriaId → Categorias; Ventas.ClienteId → Clientes; Ventas.ProductoId → Productos; Ventas.VendedorId → Vendedores. Ventas.Total = Cantidad × Productos.Precio en el momento de registrar (lo calcula el sistema, no el usuario).

Script SQL

Ejecuta el script completo desde SSMS o desde Visual Studio (conectado a `master`). El script es reejecutable: elimina y vuelve a crear la base.

```
IF DB_ID(N'Ej18_ProyectoFinal') IS NOT NULL
BEGIN
    ALTER DATABASE Ej18_ProyectoFinal SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE Ej18_ProyectoFinal;
END
GO

CREATE DATABASE Ej18_ProyectoFinal;
GO

USE Ej18_ProyectoFinal;
GO

CREATE TABLE Categorias (
    CategoriaId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre       NVARCHAR(40)  NOT NULL UNIQUE
);

CREATE TABLE Productos (
    ProductoId   INT            IDENTITY(1,1) PRIMARY KEY,
    Nombre       NVARCHAR(80)   NOT NULL,
    CategoriaId  INT            NOT NULL REFERENCES Categorias(CategoriaId),
    Precio       DECIMAL(12,2)  NOT NULL CHECK (Precio > 0),
    Stock        INT            NOT NULL CHECK (Stock >= 0)
);

CREATE TABLE Clientes (
    ClienteId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre     NVARCHAR(80)  NOT NULL,
    Ciudad     NVARCHAR(50)  NOT NULL
);

CREATE TABLE Vendedores (
    VendedorId  INT           IDENTITY(1,1) PRIMARY KEY,
    Nombre      NVARCHAR(60)  NOT NULL,
    Zona        NVARCHAR(20)  NOT NULL
);

CREATE TABLE Ventas (
    VentaId     INT            IDENTITY(1,1) PRIMARY KEY,
    ClienteId   INT            NOT NULL REFERENCES Clientes(ClienteId),
    ProductoId  INT            NOT NULL REFERENCES Productos(ProductoId),
    VendedorId  INT            NOT NULL REFERENCES Vendedores(VendedorId),
    Cantidad    INT            NOT NULL CHECK (Cantidad > 0),
    FechaVenta  DATE           NOT NULL,
    Total       DECIMAL(14,2)  NOT NULL
);

INSERT INTO Categorias (Nombre) VALUES
    (N'Tecnología'),
    (N'Hogar'),
```

```
    (N'Oficina'),
    (N'Deportes');

INSERT INTO Productos (Nombre, CategoriaId, Precio, Stock) VALUES
    (N'Mouse Óptico', 1, 35000, 60),
    (N'Teclado Mecánico', 1, 185000, 25),
    (N'Auriculares Bluetooth', 1, 85000, 40),
    (N'Monitor 24 pulgadas', 1, 690000, 10),
    (N'Licuadora 3 Velocidades', 2, 120000, 18),
    (N'Lámpara LED de Escritorio', 2, 65000, 30),
    (N'Ventilador de Torre', 2, 210000, 14),
    (N'Cafetera Eléctrica', 2, 175000, 16),
    (N'Silla Ergonómica', 3, 450000, 9),
    (N'Resma de Papel Carta', 3, 14000, 120),
    (N'Archivador Metálico', 3, 320000, 7),
    (N'Calculadora Científica', 3, 75000, 35),
    (N'Balón de Fútbol', 4, 95000, 22),
    (N'Pesas de 10 kg', 4, 140000, 15),
    (N'Colchoneta de Yoga', 4, 60000, 28),
    (N'Bicicleta Estática', 4, 980000, 4);

INSERT INTO Clientes (Nombre, Ciudad) VALUES
    (N'Andrea Molina', N'Cali'),
    (N'Bernardo Cruz', N'Bogotá'),
    (N'Carla Ramos', N'Medellín'),
    (N'Daniel Ortega', N'Cali'),
    (N'Elena Sandoval', N'Bogotá'),
    (N'Fernando Bravo', N'Barranquilla'),
    (N'Gloria Patiño', N'Medellín'),
    (N'Hugo Vanegas', N'Pereira'),
    (N'Irene Castaño', N'Cali'),
    (N'Jairo Serrano', N'Bogotá');

INSERT INTO Vendedores (Nombre, Zona) VALUES
    (N'Laura Vega', N'Norte'),
    (N'Mateo Ríos', N'Sur'),
    (N'Carolina Paz', N'Centro'),
    (N'Jorge Salas', N'Norte'),
    (N'Diana Mora', N'Sur');

INSERT INTO Ventas (ClienteId, ProductoId, VendedorId, Cantidad, FechaVenta, Total) VALUES
    (1, 10, 5, 1, '2024-03-15', 14000),
    (2, 7, 3, 1, '2024-08-05', 210000),
    (3, 13, 4, 6, '2024-10-12', 570000),
    (4, 5, 1, 4, '2024-11-28', 480000),
    (5, 1, 5, 2, '2024-12-31', 70000),
    (6, 1, 1, 2, '2025-01-10', 70000),
    (7, 13, 3, 5, '2025-03-08', 475000),
    (8, 3, 4, 1, '2025-03-22', 85000),
    (9, 8, 5, 6, '2025-04-17', 1050000),
    (10, 9, 4, 5, '2025-05-09', 2250000),
    (2, 9, 5, 1, '2025-05-30', 450000),
    (10, 16, 3, 2, '2025-07-04', 1960000),
    (5, 10, 3, 5, '2025-08-19', 70000),
    (2, 5, 2, 6, '2025-09-30', 720000),
    (6, 4, 1, 4, '2025-10-01', 2760000),
    (9, 10, 2, 5, '2025-10-18', 70000),
    (3, 16, 3, 5, '2025-11-05', 4900000),
    (8, 11, 5, 3, '2025-11-30', 960000),
    (2, 1, 1, 4, '2025-12-01', 140000),
    (8, 9, 4, 3, '2025-12-15', 1350000),
    (10, 9, 2, 6, '2025-12-31', 2700000),
    (3, 6, 5, 3, '2026-01-02', 195000),
    (7, 8, 2, 4, '2026-01-15', 700000),
    (6, 14, 5, 3, '2026-01-31', 420000),
    (9, 13, 4, 3, '2026-02-10', 285000),
    (8, 12, 3, 5, '2026-02-28', 375000),
    (3, 6, 3, 5, '2026-03-05', 325000),
    (6, 1, 2, 1, '2026-03-19', 35000),
    (6, 2, 5, 5, '2026-03-27', 925000),
    (5, 13, 5, 1, '2026-04-30', 95000),
    (8, 1, 2, 5, '2026-05-12', 175000),
    (8, 12, 3, 4, '2026-05-21', 300000),
    (9, 5, 2, 4, '2026-06-03', 480000),
    (9, 3, 3, 1, '2026-06-18', 85000),
    (4, 14, 5, 4, '2026-07-07', 560000),
    (1, 5, 4, 1, '2026-07-25', 120000),
```

```
(7, 8, 3, 3, '2026-08-01', 525000),
(10, 1, 3, 3, '2026-08-14', 105000),
(1, 16, 4, 6, '2026-08-30', 5880000),
(2, 12, 4, 4, '2026-08-31', 300000),
(10, 6, 1, 2, '2026-09-01', 130000),
(6, 16, 4, 1, '2026-09-10', 980000),
(4, 7, 4, 6, '2026-09-22', 1260000),
(2, 4, 4, 1, '2026-09-23', 690000),
(3, 6, 5, 3, '2026-09-25', 195000),
(1, 6, 3, 5, '2026-09-28', 325000),
(9, 10, 2, 5, '2026-09-30', 70000),
(6, 5, 5, 2, '2026-10-05', 240000),
(4, 5, 2, 4, '2026-11-15', 480000),
(2, 8, 3, 6, '2026-12-20', 1050000);
```

Diseño del formulario

Control	Tipo	Uso
`frmMenu` · `btnVentas`, `btnReportes`, `btnSalir`	Form + Button	Menú principal que abre los demás formularios
`frmVentas` · `cmbCliente`, `cmbProducto`, `cmbVendedor`	ComboBox	Datos de la venta (ValueMember = Id)
`frmVentas` · `txtCantidad`, `dtpFecha`	TextBox, DateTimePicker	Cantidad y fecha; lblPrecio y lblTotal muestran el precio y el total calculado
`frmVentas` · `btnGuardar`, `btnActualizar`, `btnEliminar`, `btnNuevo`	Button	CRUD
`frmVentas` · `txtBuscarCliente`, `cmbFiltroCategoria`, `cmbFiltroVendedor`, `cmbFiltroAnio`, `dtpDesde`, `dtpHasta`, `btnFiltrar`	TextBox, ComboBox, DateTimePicker, Button	Búsqueda parcial y filtros múltiples; fechas con ShowCheckBox
`frmVentas` · `dgvVentas`, `lblTotal`	DataGridView, Label	Listado con JOIN y suma del resultado
`frmReportes` · `cmbPeriodo`, `txtAnio`, `cmbMes`, `dtpReferencia`, `btnPeriodo`	ComboBox, TextBox, DateTimePicker, Button	Año específico, año/mes actual, año/mes anterior, últimos 30 días, últimos 10 meses, año y mes elegidos, año actual hasta hoy
`frmReportes` · `lblCantidad`, `lblTotal`, `lblPromedio`, `lblMinimo`, `lblMaximo`	Label	COUNT, SUM, AVG, MIN, MAX
`frmReportes` · `cmbReporte`, `txtMinimo`, `btnReporte`, `dgvReporte`	ComboBox, TextBox, Button, DataGridView	Top 5 productos, ventas por vendedor (HAVING), ventas por categoría (HAVING), ventas por cliente

Requerimientos funcionales

1. El menú principal abre frmVentas y frmReportes; cada formulario abre y cierra sus propias conexiones.
2. frmVentas lista las ventas con Cliente, Producto, Vendedor, Cantidad, Fecha y Total (INNER JOIN de 4 tablas) ordenadas por fecha descendente.
3. Al registrar una venta, el Total lo calcula el sistema: Cantidad × Precio del producto.
4. No se permite registrar una cantidad menor o igual que cero, ni superior al stock del producto.
5. Se puede actualizar la cantidad y la fecha de una venta; el Total se recalcula.
6. Se puede eliminar una venta con confirmación Sí/No.
7. Búsqueda por nombre de cliente con coincidencia parcial (LIKE).
8. Filtros múltiples combinables: categoría, vendedor, año y rango de fechas (DateTimePicker).
9. lblTotal suma las ventas mostradas.
10. Los reportes por período calculan cantidad, total, promedio, mínimo y máximo.
11. Períodos obligatorios: año elegido, año actual, año actual hasta hoy, año anterior, mes elegido, mes actual, mes anterior, últimos 30 días y últimos 10 meses.
12. Todos los períodos relativos se calculan con GETDATE() / DATEADD / DATEFROMPARTS / EOMONTH; dtpReferencia (opcional) permite simular otra fecha.
13. Reporte TOP 5 de productos por unidades vendidas.
14. Reporte de ventas por vendedor con un mínimo de ventas (GROUP BY + HAVING + COUNT).
15. Reporte de ventas por categoría con un total mínimo (GROUP BY + HAVING + SUM).
16. Todos los textos del usuario viajan como parámetros SQL.
17. Se validan los campos vacíos, números negativos o no numéricos, fechas inválidas y rangos invertidos, con mensajes claros.
18. Se manejan SqlException, errores de conexión y errores de conversión sin cerrar la aplicación.

Consultas SQL que debe realizar

Tipo	Qué debes lograr
SELECT + INNER JOIN	4 tablas en el listado.
INSERT / UPDATE / DELETE	CRUD de Ventas.
LIKE	Cliente.
WHERE múltiple	Categoría, vendedor, año, rango.
GETDATE, DATEADD, DATEFROMPARTS, EOMONTH, YEAR, MONTH	Todos los períodos.
COUNT, SUM, AVG, MIN, MAX	Estadísticas del período.
TOP 5 + ORDER BY	Productos más vendidos.
GROUP BY + HAVING	Vendedores y categorías.

Pistas (sin solución)

- Divide el trabajo: primero CRUD, luego búsquedas, luego períodos, luego reportes. Prueba cada etapa con sus casos antes de seguir.
- Crea métodos reutilizables para abrir conexión, ejecutar consultas y validar datos; no repitas bloques.
- Calcula el Total consultando el Precio del producto seleccionado (no lo tomes del usuario).
- Verifica cada consulta en SSMS antes de llevarla a C#, comparando con los resultados esperados de los casos.
- Piensa qué pasa con el stock al registrar o eliminar una venta (¿debe actualizarse? El caso CP-13 no lo exige; decide y documenta).

Validaciones

Entrada / situación	Mensaje o comportamiento esperado
Cantidad vacía / 0 / negativa / texto	«La cantidad debe ser un número entero mayor que cero.»
Cantidad mayor que el stock	«La cantidad supera el stock disponible.»
Cliente, producto o vendedor sin elegir	«Seleccione un cliente.» / «Seleccione un producto.» / «Seleccione un vendedor.»
Fecha futura al registrar	«La fecha de la venta no puede ser futura.»
Rango de fechas invertido	«La fecha inicial no puede ser mayor que la fecha final.»
Año inválido en reportes	«El año debe ser un número válido.»
Mínimo negativo o texto	«El valor mínimo debe ser un número mayor o igual que cero.»
Servidor detenido	«No se pudo conectar con la base de datos. Verifique el servidor.»

Casos de prueba

Datos: 4 categorías, 16 productos, 10 clientes, 5 vendedores y 50 ventas (2024–2026). Referencia de fecha: **30/09/2026**. Ejecuta CP-01 a CP-12 (solo lectura) ANTES de los CP-13 a CP-15, que modifican datos. Reconstruye la base con el script para repetir.

ID	Acción	Datos de entrada	Resultado esperado
CP-01	Cargar listado de ventas	Abrir frmVentas (sin filtros)	50 filas; columnas Cliente, Producto, Vendedor, Cantidad, Fecha, Total; lblTotal = $38.659.000
CP-02	Búsqueda parcial por cliente	Cliente contiene: an	29 ventas (Andrea Molina, Daniel Ortega, Elena Sandoval, Fernando Bravo, Hugo Vanegas, Jairo Serrano); total $25.174.000
CP-03	Filtros múltiples	Categoría Hogar · Año 2026 · Vendedor Carolina Paz	Cantidad 4 · Total $2.225.000 · Promedio $556.250
CP-04	Año específico	Año 2025	Cantidad 16 · Total $20.010.000 · Promedio $1.250.625 · Mínimo $70.000 · Máximo $4.900.000
CP-05	Mes actual	Ref. 30/09/2026 (01/09/2026 al 30/09/2026)	Cantidad 7 · Total $3.650.000 · Promedio $521.428,57 · Mínimo $70.000 · Máximo $1.260.000
CP-06	Mes anterior	Ref. 30/09/2026 (01/08/2026 al 31/08/2026)	Cantidad 4 · Total $6.810.000 · Promedio $1.702.500 · Mínimo $105.000 · Máximo $5.880.000
CP-07	Últimos 10 meses	Ref. 30/09/2026	Fecha inicial 01/12/2025 · fecha final 30/09/2026 · Cantidad 29 · Total $19.725.000 · Promedio $680.172,41
CP-08	Últimos 30 días	Ref. 30/09/2026	Período 31/08/2026 al 30/09/2026 · Cantidad 8 · Total $3.950.000 · Promedio $493.750
CP-09	Año anterior vs año actual hasta hoy	Ref. 30/09/2026	Año anterior (2025): Cantidad 16 · Total $20.010.000 · Promedio $1.250.625 · Año actual hasta hoy: Cantidad 26 · Total $15.535.000 · Promedio $597.500
CP-10	TOP 5 productos por unidades	Sin filtros	Licuadora 3 Velocidades (21), Cafetera Eléctrica (19), Lámpara LED de Escritorio (18), Mouse Óptico (17), Resma de Papel Carta (16)
CP-11	Vendedores con más de 11 ventas (HAVING)	Agrupar por vendedor	2 vendedores: Carolina Paz (13), Diana Mora (12)
CP-12	Categorías con total > $8.000.000 (HAVING)	Agrupar por categoría	3 categorías: Deportes ($16.125.000), Oficina ($8.909.000), Hogar ($8.485.000)
CP-13	Registrar venta (INSERT)	Andrea Molina · Monitor 24 pulgadas · Laura Vega · Cantidad 3 · 15/09/2026	«Venta registrada correctamente.»; Total calculado = 3 × $690.000 = $2.070.000; listado pasa a 51 filas; Mes actual pasa a Cantidad 8 · Total $5.720.000
CP-14	Actualizar venta (UPDATE)	Editar la venta del CP-13: Cantidad 3 → 5	«Venta actualizada correctamente.»; Total = 5 × $690.000 = $3.450.000; Mes actual: Total $7.100.000
CP-15	Eliminar venta (DELETE)	Eliminar la venta del CP-13: primero «No», luego «Sí»	Con «No»: 51 filas. Con «Sí»: «Venta eliminada correctamente.»; 50 filas; Mes actual vuelve a Cantidad 7 · Total $3.650.000

Verificación de errores (sin ID numérico): detén el servicio SQL Server o cambia el nombre del servidor y ejecuta cualquier consulta → «No se pudo conectar con la base de datos. Verifique el servidor.» sin cerrar la aplicación.

Reto adicional

Actualiza el stock del producto al registrar, modificar o eliminar una venta dentro de una **transacción** (SqlTransaction), de modo que si falla una de las dos operaciones no quede ninguna. Con el CP-13 el stock de Monitor 24 pulgadas debe pasar de 10 a 7; con el CP-14, a 5; con el CP-15 («Sí»), volver a 10.

Espacio para notas

8. Matriz de seguimiento de casos de prueba

Marca cuántos casos de prueba pasó tu programa en cada ejercicio. Un ejercicio se considera terminado cuando todos los casos pasan.

N.º	Ejercicio	Casos	Aprobados	Listo
1	SELECT Y CONEXIÓN A SQL SERVER	8	_____ / 8	☐
2	INSERT — REGISTRAR PRODUCTOS	10	_____ / 10	☐
3	UPDATE — ACTUALIZAR CLIENTES	9	_____ / 9	☐
4	DELETE — ELIMINAR PROVEEDORES	7	_____ / 7	☐
5	CRUD COMPLETO DE EMPLEADOS	12	_____ / 12	☐
6	BÚSQUEDA POR NOMBRE (LIKE)	10	_____ / 10	☐
7	BÚSQUEDA POR APELLIDO Y OTROS CAMPOS	12	_____ / 12	☐
8	FILTROS POR RANGO DE FECHAS	10	_____ / 10	☐
9	FUNCIONES DE AGREGACIÓN: SUM, COUNT, AVG, MIN Y MAX	10	_____ / 10	☐
10	TOP Y ORDER BY: RANKINGS	10	_____ / 10	☐
11	FILTROS MÚLTIPLES COMBINADOS	15	_____ / 15	☐
12	INNER JOIN: CLIENTES Y VENTAS	9	_____ / 9	☐
13	GROUP BY: ESTADÍSTICAS POR GRUPO	9	_____ / 9	☐
14	HAVING: FILTRAR GRUPOS	10	_____ / 10	☐
15	CONSULTAS POR PERÍODOS DE TIEMPO	26	_____ / 26	☐
16	REPORTES Y ESTADÍSTICAS POR PERÍODO	11	_____ / 11	☐
17	SISTEMA CON MÚLTIPLES FILTROS Y REPORTES	13	_____ / 13	☐
18	PROYECTO FINAL: SISTEMA DE GESTIÓN COMERCIAL	15	_____ / 15	☐
	TOTAL	**206**	_____ / 206	

Registro de errores frecuentes encontrados

Escribe aquí los errores que cometiste y cómo los corregiste

9. Checklist final

Marca cada concepto cuando lo hayas usado y puedas explicarlo sin mirar la guía.

☐ Conexión a SQL Server	☐ Múltiples JOIN
☐ SqlConnection	☐ GETDATE()
☐ SqlCommand	☐ DATEADD()
☐ SqlDataReader	☐ DATEDIFF()
☐ SqlDataAdapter	☐ YEAR()
☐ DataTable	☐ MONTH()
☐ DataGridView	☐ DATEFROMPARTS()
☐ SELECT	☐ EOMONTH()
☐ INSERT	☐ Año específico
☐ UPDATE	☐ Año actual
☐ DELETE	☐ Año anterior
☐ CRUD	☐ Mes específico
☐ LIKE	☐ Mes actual
☐ Parámetros SQL	☐ Mes anterior
☐ ORDER BY	☐ Últimos 7 días
☐ TOP	☐ Últimos 30 días
☐ SUM	☐ Últimos 10 meses
☐ AVG	☐ Últimos 12 meses
☐ COUNT	☐ Filtros múltiples
☐ MIN	☐ Validaciones
☐ MAX	☐ Manejo de errores
☐ GROUP BY	☐ Casos de prueba
☐ HAVING	☐ Resultados esperados
☐ INNER JOIN	☐ Proyecto final

¿Listo? Si pudiste completar el proyecto final pasando sus 15 casos de prueba, ya eres capaz de construir por tu cuenta una aplicación de escritorio completa conectada a SQL Server.

Metas para mi siguiente proyecto